

I/O·비동기 UI·다중 카메라

파일·소켓·Buffer에서 취소 가능한 이미지 검사 스레드 설계까지

-
- 01 파일·소켓·비동기 I/O와 Buffer
-
- 02 비동기 작업 취소와 UI 정지 없는 이미지 처리
-
- 03 다중 카메라 이미지 검사 스레드 설계와 멈춤 진단
-

원본 Markdown 9개의 내용·코드·표·수식을 보존하여 연관 주제별로 편집

title: 파일·소켓·비동기 I/O와 Buffer aliases:

tags:

date: 2026-08-04

- 파일과 소켓
- 비동기 IO
- Buffer 수명
- 카메라 SDK 버퍼 수명
- CS
- OS
- Cpp
- File
- Socket
- AsyncIO
- Buffer

<<<<<<< HEAD =====

origin/main

파일·소켓·비동기 I/O와 Buffer

[!summary] 이 문서의 핵심

I/O 코드는 **핸들 수명**, **요청 수명**, **버퍼 수명**, **완료 통지 수명**을 따로 추적해야 한다. 특히 비동기 I/O나 카메라 콜백에서는 포인터를 받았다는 사실이 소유권을 뜻하지 않는다. 버퍼를 작업 스레드로 넘기려면 ① 애플리케이션 소유 메모리로 복사하거나, ② SDK가 공식적으로 제공하는 retain/lock/requeue 계약을 RAII로 감싸야 한다.

0. 문서 범위와 기존 문서와의 관계

이 문서는 다음 내용을 한 흐름으로 연결한다.

- 파일과 파일 디스크립터
- 소켓과 준비 상태(readiness)
- blocking, non-blocking, asynchronous I/O의 차이
- I/O 요청이 참조하는 버퍼의 수명
- 콜백 버퍼를 검사 스레드로 전달하는 방법
- 카메라 SDK의 버퍼 재사용·반납 계약
- 안전한 종료 순서

다음 일반 개념은 기존 문서에서 이미 자세히 다뤘으므로 여기서는 I/O와 직접 연결되는 부분만 사용한다.

- 메모리·RAII·스마트 포인터: 00.메모리·객체 수명·RAII
- 프로세스·스레드·파일·소켓 개요: 01.프로세스와 스레드
- 동시성·비동기성·이벤트 루프: 02.동시성과 병렬성
- bounded queue·조건 변수·종료 프로토콜: 03.Mutex와 Condition Variable
- 객체 수명과 소유권 보완: 04.C++ 메모리와 객체 수명 보완

1. PDF 대조 범위와 해석 한계

세 PDF에서 다음 내용을 대조했다.

자료	확인한 핵심 내용	PDF 위치
OSTEP	non-blocking I/O와 <code>select()</code> / <code>poll()</code> , 이벤트 기반 서버, POSIX AIO의 요청 구조체와 버퍼 주소, 파일 디스크립터, <code>read</code> / <code>write</code> , write buffering, DMA	물리 398~404, 411~415, 463~469쪽 부근
OCS	socket endpoint, TCP/UDP, interrupt-driven I/O, DMA의 메모리 주소·길이와 전송 중 메모리 제약, 커널 I/O 하위 시스템	물리 109~112, 480, 485, 490~500쪽 부근
C++ PDF	blocking 파일 읽기와 작업 스레드, <code>promise</code> / <code>future</code> 를 통한 결과 전달, thread pool의 stop-notify-join	물리 797~822쪽 부근

[!warning] 카메라 SDK에 대한 한계

세 PDF는 특정 카메라 제조사의 SDK 계약을 정의하지 않는다. 따라서 “콜백 포인터가 언제까지 유효한가”, “반납 함수가 무엇인가”, “SDK가 프레임을 언제 재사용하는가”는 PDF만으로 확인할 수 없다. 이 문서에서는 PDF에서 확인한 일반 I/O·수명 원칙을 적용하되, **제조사 문서에서 반드시 확인할 항목**을 명시한다.

2. 한 번의 I/O에는 네 가지 수명이 있다

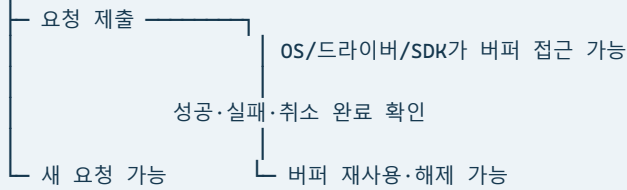
I/O 문제를 “포인터 하나”로 보면 수명 오류가 생긴다. 다음 네 대상을 분리해서 봐야 한다.

대상	예	종료 조건
핸들(handle)	파일 디스크립터, socket, camera handle	<code>close</code> , SDK close/destroy가 완료될 때
요청(operation/request)	<code>read</code> , overlapped request, AIO control block, frame acquisition	성공·실패·취소의 완료 가 확인될 때
버퍼(storage)	<code>std::vector<std::byte></code> , SDK frame memory, DMA 대상 메모리	마지막 접근 주체가 사용을 끝낼 때
완료 통지(completion)	callback, event, future, completion queue item	소비자가 결과를 인수하고 필요한 후처리를 끝낼 때

이 관계는 다음처럼 이해할 수 있다.

TEXT

핸들 열기



핸들 닫기는 진행 중인 요청과 콜백이 모두 정리되는 시점과 조정해야 한다.

가장 중요한 규칙은 다음과 같다.

요청을 제출한 함수가 반환한 시점이 아니라, 그 요청의 완료 또는 취소 완료가 확인된 시점까지 요청 객체와 버퍼가 살아 있어야 한다.

단, 동기 `read()` 처럼 함수 반환 자체가 해당 호출의 완료를 뜻하는 API는 반환 뒤 버퍼를 사용할 수 있다. API마다 “완료”의 정의가 다르므로 계약을 확인해야 한다.

3. 파일 I/O

3.1 파일 디스크립터는 파일 자체가 아니다

`open()` 은 프로세스가 사용하는 파일 디스크립터를 반환한다. 파일 디스크립터는 열린 파일을 가리키는 정수형 핸들이며, 커널의 열린 파일 상태와 연결된다.

CPP

```

#include <cerrno>
#include <cstddef>
#include <span>
#include <system_error>
#include <unistd.h>

std::size_t read_some(int fd, std::span<std::byte> out) {
    for (;;) {
        const auto n = ::read(fd, out.data(), out.size());
        if (n > 0) {
            return static_cast<std::size_t>(n);
        }
        if (n == 0) {
            return 0; // 일반 파일에서는 EOF
        }
        if (errno == EINTR) {
            continue; // signal로 중단되었으므로 재시도
        }
        throw std::system_error(errno, std::generic_category(), "read");
    }
}

```

`read()` 의 반환값은 “요청한 크기”가 아니라 **실제로 읽은 바이트 수**다. 따라서 버퍼 전체가 채워졌다고 가정하면 안 된다.

3.2 부분 읽기와 부분 쓰기

파일과 소켓 모두 한 번의 호출이 요청량 전체를 처리한다는 보장이 없는 경우가 있다. 특히 소켓, pipe, non-blocking 핸들에서는 부분 처리가 정상적인 흐름이다.

C++

```
#include <cerrno>
#include <cstdint>
#include <span>
#include <system_error>
#include <unistd.h>

void write_all(int fd, std::span<const std::byte> data) {
    std::size_t offset = 0;

    while (offset < data.size()) {
        const auto* begin = data.data() + offset;
        const auto left = data.size() - offset;
        const auto n = ::write(fd, begin, left);

        if (n > 0) {
            offset += static_cast<std::size_t>(n);
            continue;
        }
        if (n < 0 && errno == EINTR) {
            continue;
        }
        throw std::system_error(errno, std::generic_category(), "write");
    }
}
```

[!note] `write()` 성공과 영구 저장은 다르다

`write()` 가 성공해도 데이터는 메모리의 커널 버퍼에만 있을 수 있다. 전원 장애 뒤에도 유지되는 저장까지 필요하다면 파일 시스템과 API 계약에 따라 `fsync()` 같은 동기화가 추가로 필요하다. 즉 “애플리케이션 버퍼에서 커널이 인수함”과 “저장 장치에 영구 반영됨”을 구분해야 한다.

3.3 파일 핸들을 RAII로 관리하기

CPP

```
#include <unistd.h>
#include <utility>

class UniqueFd {
public:
    UniqueFd() noexcept = default;
    explicit UniqueFd(int fd) noexcept : fd_(fd) {}

    ~UniqueFd() {
        reset();
    }

    UniqueFd(const UniqueFd&) = delete;
    UniqueFd& operator=(const UniqueFd&) = delete;

    UniqueFd(UniqueFd&& other) noexcept
        : fd_(std::exchange(other.fd_, -1)) {}

    UniqueFd& operator=(UniqueFd&& other) noexcept {
        if (this != &other) {
            reset(std::exchange(other.fd_, -1));
        }
        return *this;
    }

    [[nodiscard]] int get() const noexcept { return fd_; }

    void reset(int next = -1) noexcept {
        if (fd_ != -1) {
            ::close(fd_);
        }
        fd_ = next;
    }

private:
    int fd_ = -1;
};
```

`UniqueFd` 는 핸들의 소유권만 표현한다. 진행 중인 비동기 요청이 끝났다는 사실까지 보장하지는 않는다. 비동기 API에 서는 핸들 RAII 바깥에 요청 취소·완료 대기 프로토콜이 추가되어야 한다.

4. 소켓 I/O

4.1 소켓은 통신의 endpoint다

소켓은 통신의 한쪽 끝을 나타내는 핸들이다. TCP는 연결을 만들고 바이트 흐름을 주고받는 방식이며, UDP는 개별 datagram을 주고받는 방식이다.

항목	TCP	UDP
연결	연결 기반	비연결형 사용 가능
데이터 관점	연속된 바이트 스트림	datagram 단위
애플리케이션 경계	별도 framing 필요	datagram 경계가 존재

항목	TCP	UDP
일반적인 처리	부분 <code>send</code> / <code>recv</code> , 연결 종료 처리	datagram 크기·유실·순서 정책 처리

TCP의 한 번의 `recv()` 결과를 하나의 완성된 메시지로 간주하면 안 된다. 길이 prefix, 구분자, 고정 길이처럼 애플리케이션 프로토콜이 메시지 경계를 정의해야 한다.

4.2 준비 상태는 작업 완료와 다르다

`select()` / `poll()` / 플랫폼별 이벤트 API가 알려 주는 것은 보통 "지금 해당 연산을 시도하면 block되지 않을 가능성이 있는 상태"다.

TEXT

`readable` 알림

- ├ 읽을 데이터가 있음
- ├ `peer`가 정상 종료하여 EOF를 읽을 수 있음
- └ 오류 상태를 관찰할 수 있음

`readable` ≠ 애플리케이션 메시지 하나가 완전히 도착함

따라서 이벤트 루프에는 다음 상태가 함께 있어야 한다.

- 연결별 수신 버퍼
- 현재까지 받은 바이트 수
- 다음 메시지에 필요한 길이
- 부분 전송의 offset
- 연결 종료 및 오류 상태

4.3 non-blocking은 asynchronous와 같지 않다

방식	호출자가 기다리는가?	실제 작업 완료를 어떻게 아는가?	버퍼 수명
blocking	호출 스레드가 기다림	함수 반환	반환까지 유효
non-blocking	즉시 반환	현재 처리량 또는 <code>EAGAIN</code> / <code>EWOULDBLOCK</code> ; readiness 뒤 재시도	각 호출이 접근하는 동안 유효
readiness 기반	이벤트 대기과 실제 <code>read</code> / <code>write</code> 가 분리	알림 뒤 호출하여 처리	연결별 누적 상태가 계속 유효해야 함
completion 기반 async	요청을 제출하고 반환	callback/event/completion queue/future	완료 또는 취소 완료까지 유효

[!important] `std::async` 와 OS 비동기 I/O

`std::async([&] { blocking_read(); })` 는 blocking 호출을 다른 실행 주체에서 수행하게 만들 수 있지만, 그 자체가 커널의 completion 기반 비동기 I/O로 바뀌는 것은 아니다. C++ 작업 비동기화와 운영체제 I/O 모델을 구분해야 한다.

5. 비동기 I/O 요청과 버퍼

5.1 제출 시점부터 완료 시점까지

OSTEP의 POSIX AIO 예에서는 요청 구조체에 파일 디스크립터, offset, 길이와 함께 버퍼 위치가 들어간다. OS가 나중에 그 위치에 데이터를 기록하므로, 함수의 지역 변수가 먼저 소멸하면 안 된다.

CPP

```
// 잘못된 구조를 보여 주는 개념 예시
void start_bad_async_read(AsyncFile& file) {
    std::array<std::byte, 4096> local{};
    file.async_read(local.data(), local.size(), on_complete);
} // local 소멸: 완료 전이라면 dangling buffer
```

안전한 요청 객체는 버퍼와 완료 상태를 함께 소유한다.

CPP

```
#include <cstddef>
#include <memory>
#include <vector>

struct ReadOperation {
    std::vector<std::byte> buffer;
    std::size_t transferred = 0;
    // 플랫폼별 요청 객체, offset, 오류 상태 등을 함께 둔다.
};

void start_read(AsyncFile& file, std::size_t size) {
    auto operation = std::make_shared<ReadOperation>();
    operation->buffer.resize(size);

    file.async_read(
        operation->buffer.data(),
        operation->buffer.size(),
        [operation](std::error_code ec, std::size_t n) {
            operation->transferred = n;
            if (!ec) {
                consume(std::span(operation->buffer).first(n));
            }
        });
}
```

이 예의 `shared_ptr` 는 “무조건 비동기는 shared ownership”이라는 뜻이 아니다. API의 callback 저장 방식에 맞춰 요청 객체의 수명을 완료까지 연장하는 한 방법이다. 단일 소유 이벤트 루프라면 `unique_ptr` 요청을 completion map에 보관하는 편이 더 명확할 수 있다.

5.2 취소 요청은 즉시 해제가 아니다

일반적인 안전 규칙은 다음과 같다.

1. 취소를 **요청**한다.
2. completion 또는 별도의 동기화 수단으로 취소 **완료**를 확인한다.
3. OS-드라이버-SDK가 더 이상 버퍼에 접근하지 않는다는 계약을 만족한다.
4. 그 뒤 요청 객체와 버퍼를 재사용하거나 해제한다.

`cancel()` 이 반환했다는 이유만으로 즉시 버퍼를 해제할 수 있는지는 API 계약에 달려 있다. "취소 요청 접수"와 "접근 종료"를 혼동하면 use-after-free가 된다.

5.3 DMA가 보여 주는 본질

DMA에서는 장치가 CPU 대신 메모리 위치와 길이를 이용해 데이터를 전송한다. 전송이 끝나기 전에 해당 메모리를 수정·이동·해제하면 장치와 프로그램이 같은 저장 공간을 잘못 공유하게 된다.

이 원리는 카메라 프레임에도 그대로 적용된다.

- SDK/드라이버가 버퍼를 채우는 동안 애플리케이션이 덮어쓰면 안 된다.
- 검사 스레드가 읽는 동안 SDK가 같은 버퍼를 다음 프레임용으로 재사용하면 안 된다.
- 재사용이 필요하다면 소유권 이전, retain/lock, reference count, requeue 같은 명시적 프로토콜이 있어야 한다.

6. Buffer: 소유 메모리와 빌린 view를 구분하기

6.1 타입으로 표현하기

CPP

```
#include <cstdint>
#include <cstdint>
#include <span>
#include <vector>

struct BorrowedFrameView {
    std::span<const std::byte> pixels; // 소유하지 않음
    std::uint32_t width{};
    std::uint32_t height{};
    std::uint32_t stride{};
    std::uint64_t sequence{};
};

struct OwnedFrame {
    std::vector<std::byte> pixels; // 애플리케이션이 소유
    std::uint32_t width{};
    std::uint32_t height{};
    std::uint32_t stride{};
    std::uint64_t sequence{};
};
```

- `std::span` 은 범위를 표현하지만 수명을 연장하지 않는다.
- `std::vector` 는 자기 저장 공간을 소유하고 이동할 수 있다.
- raw pointer와 `span` 이 존재한다고 해서 그 메모리를 `delete` 할 권리가 생기지 않는다.

6.2 실제 소유자를 판별하는 질문

버퍼의 실제 소유자는 "현재 포인터를 들고 있는 코드"가 아니라 다음 계약으로 결정된다.

1. 누가 메모리를 할당했는가?
2. 누가 언제 재사용할 수 있는가?
3. 누가 마지막 사용을 선언하는가?
4. 어떤 함수로 반납·해제해야 하는가?
5. retain/lock/reference count를 증가시킬 수 있는가?
6. callback 반환이 자동 반납을 의미하는가?

생성 방식	일반적인 소유자	해제/반납 원칙
애플리케이션의 <code>new[]</code>	애플리케이션	대응되는 <code>delete[]</code>
<code>std::vector</code> 내부 저장 공간	해당 <code>vector</code>	자동 소멸; <code>data()</code> 에 <code>delete</code> 금지
C <code>malloc</code>	할당 계약의 소유자	<code>free</code>
SDK가 만든 frame/buffer	SDK 또는 SDK에서 lease를 받은 앱	SDK 전용 <code>release/requeue</code> 함수
앱이 SDK에 등록한 buffer	저장 공간은 앱 소유, 사용 권한은 작업 중 SDK와 조정	완료 전 수정·해제·unregister 금지

[!danger] 할당자와 해제자를 섞지 않는다

SDK가 준 포인터에 임의로 `delete`, `delete[]`, `free` 를 호출하면 안 된다. C++에서 해제 함수는 포인터 타입이 아니라 할당·반납 계약으로 결정된다.

7. 카메라 콜백 버퍼: 세 가지 계약

카메라 SDK 이름을 모르는 상태에서는 다음 세 모델 중 어느 것인지 먼저 확인해야 한다.

7.1 모델 A — callback 동안만 빌린 버퍼

TEXT

SDK가 프레임 버퍼 소유
 → callback에 view 전달
 → callback 반환
 → SDK가 즉시 재사용 가능

이 모델에서는 검사 스레드에 pointer나 `span` 만 넘기면 안 된다. callback 안에서 애플리케이션 소유 버퍼로 복사한 뒤 그 소유 객체를 queue로 이동해야 한다.

장점은 수명 설계가 단순하고 SDK 종료와 검사 작업의 결합이 약하다는 것이다. 단점은 프레임당 메모리 복사 비용과 할당 비용이다. 할당 비용은 미리 만든 buffer pool로 줄일 수 있다.

7.2 모델 B — retain/lock/requeue 가능한 SDK frame

TEXT

SDK frame 획득
 → retain/lock 또는 소유 handle 인수
 → callback 반환 뒤에도 검사 스레드가 사용
 → 검사 완료
 → SDK `release/requeue`

SDK가 이 계약을 명시적으로 제공한다면 복사를 피할 수 있다. 대신 다음 제약이 생긴다.

- 모든 retained frame을 반납하기 전 카메라-stream-SDK 객체를 파괴하면 안 된다.
- 검사 지연이 길면 SDK buffer pool이 고갈될 수 있다.
- queue 크기를 제한하고 drop/backpressure 정책을 정해야 한다.
- 반납 함수는 RAII deleter에 정확히 연결해야 한다.

7.3 모델 C — 애플리케이션이 버퍼를 등록

애플리케이션이 버퍼를 할당해 SDK에 등록했다라도, 제출 뒤 완료 전까지는 마음대로 재사용하거나 해제할 수 없다. 저장 공간의 소유권과 특정 기간의 접근 권한은 별개다.

TEXT

앱이 **buffer** 소유

- SDK에 등록/queue
- 카메라·DMA가 기록
- 완료 callback
- 앱이 처리
- 다시 SDK에 **requeue**

동일한 버퍼를 requeue한 순간부터 다음 완료까지 검사 스레드가 계속 읽으면 race가 된다. requeue 전에 검사를 끝내거나, 별도 버퍼로 복사하거나, SDK가 허용하는 다중 버퍼 pool을 운영해야 한다.

8. 질문별 결론

8.1 버퍼의 실제 소유자는 누구인가?

포인터를 전달받은 쪽이 아니라 **할당·retain·release 계약에 기록된 주체**다. callback 인수의 `const void*` 나 `unsigned char*` 만으로는 소유권을 알 수 없다. SDK 문서에서 allocation, lifetime, ownership, buffer handling, requeue, release 항목을 확인한다.

8.2 콜백이 끝난 뒤에도 버퍼가 유효한가?

SDK 문서가 보장할 때만 유효하다. 보장이 없다면 callback 동안만 유효한 borrowed view로 취급하는 것이 안전하다. callback이 끝난 뒤 사용할 때는 다음 중 하나가 필요하다.

- callback 안에서 app-owned buffer로 복사
- SDK의 retain/lock API로 수명 연장
- 소유권이 포함된 frame handle을 인수하고 나중에 SDK 함수로 반납

8.3 검사 스레드로 넘길 때 복사가 필요한가?

조건	복사 필요 여부
callback 동안만 유효	필요
SDK가 callback 반환 뒤 즉시 재사용 가능	필요
공식 retain/lock 후 worker가 사용 가능	필수 아님
소유 frame handle을 worker에게 이동 가능	필수 아님
app buffer이지만 requeue 즉시 SDK가 덮어씀	requeue 전에 검사 완료하거나 복사 필요

복사를 피하는 것이 항상 더 빠른 것은 아니다. zero-copy로 인해 SDK buffer pool이 고갈되어 촬영이 멈추거나 프레임이 누락되면 전체 처리량은 더 나빠질 수 있다.

8.4 이미지 처리 중 카메라 SDK가 버퍼를 재사용하지 않는가?

일반 PDF만으로는 특정 SDK의 동작을 확정할 수 없다. ring buffer, buffer pool, callback 반환 시 자동 반납, 명시적 requeue 중 어떤 모델인지 제조사 문서로 확인해야 한다.

확인 전에는 다음 보수적 규칙을 사용한다.

callback에서 받은 borrowed buffer는 callback 반환 뒤 SDK가 재사용할 수 있다고 보고, 장기 작업에는 pointer만 저장하지 않는다.

8.5 해제 함수는 delete 인가, SDK 전용 함수인가?

- `new T` 로 직접 만들었다면 `delete`
- `new T[]` 로 직접 만들었다면 `delete[]`
- `malloc` 계열이면 `free`
- SDK가 생성·획득·대여한 frame이면 문서가 정한 SDK `release` / `unlock` / `requeue`
- `vector.data()` 는 직접 해제하지 않음

함수 이름이 `GetBuffer()` 라고 해서 호출자가 `delete` 할 수 있는 것은 아니다.

8.6 종료 시 카메라보다 검사 스레드를 먼저 종료해야 하는가?

“카메라 종료”를 두 단계로 나눠야 답이 명확하다.

1. **새 프레임 생산 중지**: 검사 스레드 종료보다 먼저 한다.
2. **카메라/stream/SDK 객체 파괴**: outstanding SDK frame이 모두 반납되고 검사 스레드가 종료된 뒤 한다.

권장 순서는 다음과 같다.

TEXT

새 작업 접수 중지

- 카메라 acquisition 중지 / callback 등록 해제 시작
- 실행 중 callback과 I/O의 완료·취소 완료 대기
- frame queue close
- 검사 스레드가 drain 또는 명시적 discard 후 종료(join)
- outstanding SDK frame 전부 release/requeue
- camera/stream/SDK destroy

SDK frame을 worker가 직접 보유한다면 카메라를 먼저 파괴해서는 안 된다. 반대로 callback에서 완전히 복사한 app-owned frame만 queue에 있다면 worker는 카메라 메모리에 의존하지 않지만, 새 callback과 종료가 경쟁하지 않도록 acquisition 중지와 callback quiescence는 여전히 먼저 수행해야 한다.

9. C++ 설계 1 — callback에서 복사해 소유권 넘기기

다음 예시는 특정 SDK API가 아니라 수명 구조를 보여 주는 C++20 예제다.

CPP · 1/3

```
#include <algorithm>
#include <condition_variable>
#include <cstddef>
#include <cstdint>
#include <deque>
#include <mutex>
#include <optional>
#include <span>
#include <thread>
#include <utility>
#include <vector>

struct BorrowedFrameView {
    std::span<const std::byte> pixels;
    std::uint32_t width{};
    std::uint32_t height{};
    std::uint32_t stride{};
    std::uint64_t sequence{};
};

struct OwnedFrame {
    std::vector<std::byte> pixels;
    std::uint32_t width{};
    std::uint32_t height{};
    std::uint32_t stride{};
    std::uint64_t sequence{};
};

OwnedFrame copy_frame(BorrowedFrameView source) {
    return OwnedFrame{
        std::vector<std::byte>(source.pixels.begin(), source.pixels.end()),
        source.width,
        source.height,
        source.stride,
        source.sequence
    };
}

class FrameQueue {
public:
    explicit FrameQueue(std::size_t capacity) : capacity_(capacity) {}
```

CPP · 2/3

```

// callback을 오래 막지 않도록 가득 차면 false를 반환한다.
bool try_push(OwnedFrame frame) {
    {
        std::lock_guard lock(mutex_);
        if (closed_ || queue_.size() == capacity_) {
            return false;
        }
        queue_.push_back(std::move(frame));
    }
    not_empty_.notify_one();
    return true;
}

std::optional<OwnedFrame> pop() {
    std::unique_lock lock(mutex_);
    not_empty_.wait(lock, [this] {
        return closed_ || !queue_.empty();
    });

    if (queue_.empty()) {
        return std::nullopt;
    }

    OwnedFrame frame = std::move(queue_.front());
    queue_.pop_front();
    return frame;
}

void close() {
    {
        std::lock_guard lock(mutex_);
        closed_ = true;
    }
    not_empty_.notify_all();
}

private:
    const std::size_t capacity_;
    std::mutex mutex_;
    std::condition_variable not_empty_;
    std::deque<OwnedFrame> queue_;
    bool closed_ = false;
};

```

CPP · 3/3

```

void inspect(const OwnedFrame& frame) {
    // pixels는 OwnedFrame이 소유하므로 이 함수가 실행되는 동안 유효하다.
    [[maybe_unused]] const auto size = frame.pixels.size();
}

class InspectionPipeline {
public:
    explicit InspectionPipeline(std::size_t queue_capacity)
        : queue_(queue_capacity), worker_( [this] { worker_loop(); } ) {}

    // SDK callback 안에서 호출한다고 가정한다.
    void on_frame(BorrowedFrameView borrowed) {
        // callback 반환 전에 복사를 끝낸다.
        // 실패 시 drop-newest 정책을 사용한다.
        [[maybe_unused]] const bool accepted =
            queue_.try_push(copy_frame(borrowed));
    }

    void shutdown() {
        // 실제 프로그램에서는 이 함수 전에 다음을 수행한다.
        // 1) SDK acquisition 중지
        // 2) callback 진입 차단 및 실행 중 callback 종료 대기
        queue_.close();
        if (worker_.joinable()) {
            worker_.join();
        }
        // 그 뒤 camera/stream/SDK 객체를 파괴한다.
    }

    ~InspectionPipeline() {
        shutdown();
    }

    InspectionPipeline(const InspectionPipeline&) = delete;
    InspectionPipeline& operator=(const InspectionPipeline&) = delete;

private:
    void worker_loop() {
        while (auto frame = queue_.pop()) {
            inspect(*frame);
        }
    }

    FrameQueue queue_;
    std::thread worker_;
};

```

이 구조의 핵심은 callback의 **BorrowedFrameView** 가 queue에 들어가지 않는다는 점이다. queue에는 오직 **OwnedFrame** 만 들어가며, 이동 후에는 worker가 그 객체의 유일한 소유자가 된다.

9.1 복사형 설계에서 추가로 정할 정책

- queue가 찼을 때 callback을 block할지, newest/oldest frame을 버릴지
- 이미지 크기 변화가 가능한지
- `stride * height` 와 실제 payload 크기를 어떻게 검증할지
- 프레임 번호 누락을 어떻게 기록할지
- 매 프레임 할당을 피하기 위해 buffer pool을 쓸지
- 검사 실패와 카메라 연결 해제를 어떤 상태 머신으로 처리할지

카메라 callback을 오래 block하면 SDK 내부 buffer 반환이 늦어질 수 있으므로, 실시간 시스템에서는 짧은 복사-enqueue 뒤 빠르게 반환하는 설계가 보통 이해하기 쉽다. 단, 최종 정책은 SDK의 callback threading 규칙과 허용 blocking 시간을 확인해야 한다.

10. C++ 설계 2 — SDK frame을 RAII lease로 넘기기

SDK가 callback 이후 사용 가능한 frame handle과 전용 release 함수를 공식 제공할 때만 적용한다.

CPP

```
#include <memory>
#include <utility>

struct SdkFrame; // SDK가 정의하는 불투명 frame이라고 가정

// 실제 코드에서는 정확한 SDK 함수와 필요한 camera/stream context를 보관한다.
void sdk_release_frame(SdkFrame* frame) noexcept;

struct SdkFrameReleaser {
    void operator()(SdkFrame* frame) const noexcept {
        if (frame != nullptr) {
            sdk_release_frame(frame); // delete가 아님
        }
    }
};

using FrameLease = std::unique_ptr<SdkFrame, SdkFrameReleaser>;

void enqueue_for_inspection(FrameLease frame) {
    // queue.push(std::move(frame));
    // 검사 완료 또는 queue에서 폐기될 때 SDK 전용 releaser가 정확히 한 번 실행된다.
}
```

`FrameLease` 는 다음을 표현한다.

- 복사 금지: 하나의 frame을 두 주체가 중복 반납하지 않음
- 이동 가능: callback에서 queue, queue에서 worker로 소유권 이전
- scope 종료 시 자동 반납
- `delete` 대신 SDK 전용 함수를 호출

하지만 `unique_ptr` 만으로는 충분하지 않다. deleter가 사용하는 camera/stream context가 frame보다 오래 살아 있어야 한다. 따라서 객체 파괴 순서는 보통 다음 의존성을 따라야 한다.

TEXT

FrameLease들 → FrameQueue/Workers → Stream/Camera → SDK runtime

왼쪽 객체를 먼저 정리하고 오른쪽 객체를 나중에 파괴한다.

11. Buffer pool과 backpressure

프레임 생성 속도가 검사 속도보다 빠르면 메모리를 무한히 늘려 해결할 수 없다.

정책	장점	위험/비용
producer block	프레임 보존	callback 또는 카메라 pipeline 정지 가능
drop newest	기존 순서 유지	최신 프레임 유실
drop oldest	최신 상태 우선	진행 중 판단과 시간 순서 정책 필요
camera trigger 감속	근본적인 입력률 제어	장비-공정 요구와 조정 필요
고정 buffer pool	할당 변동 감소, 메모리 상한 명확	pool 고갈 정책 필요

안전한 buffer pool의 버퍼 상태는 최소한 다음처럼 관리한다.

TEXT

Free → Filling by SDK → Ready → Inspecting → Free

하나의 버퍼가 동시에 **Filling** 과 **Inspecting** 상태에 있으면 설계 오류다. 상태 전이는 mutex/atomic/SDK requeue 계약 중 하나로 동기화해야 한다.

12. 오류 처리와 종료 상태 머신

단순한 **bool running** 하나보다 상태를 나누면 종료 경쟁을 줄일 수 있다.

TEXT

Running

- StopRequested
- AcquisitionStopped
- CallbacksQuiesced
- QueueClosed
- WorkersJoined
- FramesReleased
- CameraDestroyed

종료 구현에서 확인할 불변식은 다음과 같다.

- `QueueClosed` 뒤 새 frame이 enqueue되지 않는다.
- `WorkersJoined` 뒤 검사 함수가 frame에 접근하지 않는다.
- `FramesReleased` 뒤 outstanding SDK frame 수가 0이다.
- `CameraDestroyed` 시점에는 callback과 deleter가 camera context를 사용하지 않는다.
- destructor는 가능하면 예외를 던지지 않는다.
- 종료 함수가 두 번 호출되어도 double close/double release가 생기지 않는다.

[!warning] callback 안에서 join하지 않는다

SDK callback을 실행하는 스레드가 자기 자신 또는 자신이 빠져나가야 끝나는 작업을 `join` 하면 deadlock이 된다. 종료 요청과 실제 join 위치를 분리하고, SDK가 어느 스레드에서 callback을 실행하는지 확인한다.

13. 흔한 실패 패턴

13.1 callback pointer만 queue로 전달

CPP

```
void on_frame(const std::byte* data, std::size_t size) {
    queue.push(std::span(data, size)); // callback 뒤 SDK 재사용이면 dangling/데이터 변조
}
```

`span` 복사는 데이터 복사가 아니다. 주소와 길이만 복사한다.

13.2 지역 버퍼로 비동기 요청

CPP

```
void request() {
    std::vector<std::byte> bytes(4096);
    async_read(bytes.data(), bytes.size());
} // 완료 전에 bytes 소멸 가능
```

요청 객체가 버퍼를 소유하도록 만들고 completion까지 보관해야 한다.

13.3 SDK 포인터에 `delete[]`

CPP

```
void on_frame(unsigned char* sdk_buffer) {
    // ...
    delete[] sdk_buffer; // SDK 계약 없이는 금지
}
```

allocator mismatch, pool 손상, double free가 될 수 있다.

13.4 카메라부터 파괴

TEXT

```
camera.destroy()
worker.join() // worker가 SDK frame/deleter를 사용하면 너무 늦음
```

먼저 생산과 callback을 멈추고, worker가 frame을 모두 반납한 뒤 카메라를 파괴해야 한다.

13.5 unbounded queue

검사보다 촬영이 빠른 상황에서 queue가 무한히 늘면 지연과 메모리 사용량이 함께 증가한다. 메모리가 다 찬 뒤의 장애 대신, 미리 정한 drop/backpressure 정책으로 부하를 제한해야 한다.

14. SDK 문서에서 반드시 찾을 문장

실제 카메라 SDK를 적용할 때 아래 항목을 체크리스트로 사용한다.

- ☐ callback buffer의 유효 기간이 명시되어 있는가?
- ☐ callback 반환 시 자동으로 buffer가 재사용되는가?
- ☐ frame retain/lock API가 있는가?
- ☐ release/unlock/requeue 중 정확히 어떤 함수를 호출해야 하는가?
- ☐ 같은 thread에서만 release해야 하는가?
- ☐ callback 안에서 허용되는 작업과 금지되는 blocking 호출은 무엇인가?
- ☐ buffer alignment, stride, pixel format, payload size 규칙은 무엇인가?
- ☐ acquisition stop이 진행 중 callback의 종료까지 기다리는가?
- ☐ unregister callback 뒤 이미 진입한 callback이 남을 수 있는가?
- ☐ cancel/stop 완료를 확인하는 API가 있는가?
- ☐ camera/stream handle이 frame release보다 오래 살아야 하는가?
- ☐ SDK 종료 전 outstanding buffer를 조회할 수 있는가?

문서가 모호하면 SDK 지원팀에 다음처럼 구체적으로 질문한다.

callback으로 받은 image pointer를 callback 반환 뒤 다른 스레드에서 읽어도 되는가? 가능하다면 어떤 retain 함수가 필요하며, 어느 함수로 언제 반납해야 하는가? acquisition stop과 camera destroy는 outstanding frame의 반납을 기다리는가?

15. 설계 선택표

상황	권장 전달 방식	종료 시 핵심
SDK 수명 계약을 아직 모름	callback 안에서 app buffer로 복사	callback 종료 대기 후 queue close
callback 동안만 유효	복사 후 OwnedFrame 이동	app-owned frame drain 뒤 worker join
공식 retain/release 제공	move-only FrameLease	모든 lease 반납 후 camera destroy
앱 등록 buffer + requeue	상태가 있는 pool 또는 복사	Inspecting 버퍼가 0일 때 unregister/destroy
검사보다 입력이 빠름	bounded queue + 명시적 drop/backpressure	종료 시 정책대로 drain 또는 discard
내구성이 필요한 파일 저장	partial write 처리 + 필요 시 sync	close만으로 요구 충족 여부 검토

16. 최종 점검 요약

1. raw pointer, `span`, 파일 디스크립터는 기본적으로 소유권을 자동 표현하지 않는다.
2. blocking I/O는 함수 반환이 해당 호출의 완료지만, completion 기반 async I/O는 완료 취소 완료까지 버퍼를 유지해야 한다.
3. readiness 알림은 완성된 메시지 하나가 도착했다는 뜻이 아니다.
4. callback 뒤 버퍼 유효성은 SDK 계약 없이는 가정하지 않는다.
5. borrowed callback buffer를 검사 스레드로 넘길 때는 복사하거나 공식 frame lease를 인수한다.
6. SDK가 만든 버퍼는 `delete` 하지 않고 정확한 SDK release/requeue 함수를 사용한다.
7. 종료 때는 새 생산을 먼저 멈추고 callback을 정지시킨 뒤 queue를 닫고 worker를 join한다.
8. SDK frame이 모두 반납된 뒤 camera/stream/SDK를 파괴한다.
9. queue와 buffer pool에는 용량 상한과 overload 정책이 있어야 한다.
10. 특정 카메라에 적용하기 전 제조사 문서의 lifetime-threading-shutdown 계약을 다시 확인한다.

17. 작성 후 2차 검증 결과

17.1 Markdown 자체 검증

- UTF-8로 저장되었으며 frontmatter와 제목을 확인했다.
- 코드 fence의 시작과 끝이 모두 대응되는지 확인했다.
- 00.메모리-객체 수명·RAII부터 04.C++ 메모리와 객체 수명 보완까지의 내부 링크 대상이 실제로 존재하는지 확인했다.
- 사용자가 요청한 여섯 가지 카메라 버퍼 질문이 모두 별도 절로 존재하는지 확인했다.
- 미완성 작업을 나타내는 임시 표식이 남지 않았는지 확인했다.
- 9장의 소유 버퍼 전달 예제를 MSVC의 C++20 모드, 경고 수준 `/W4` 에서 컴파일했다.

17.2 PDF와 재대조

문서의 주장	다시 확인한 PDF 근거	판정
소켓은 통신 endpoint다	OCS 물리 109쪽(책 162쪽)	일치
<code>select()</code> 는 descriptor의 읽기·쓰기 준비 상태를 검사한다	OSTEP 물리 397~399쪽	일치
AIO 요청은 buffer 위치와 transfer 길이를 보유하고 완료를 별도로 확인한다	OSTEP 물리 400~402쪽	일치
DMA는 메모리 주소와 바이트 수를 바탕으로 전송하고 완료를 interrupt로 알린다	OSTEP 물리 414~415쪽, OCS 물리 485쪽(책 546쪽)	일치
file descriptor와 <code>read</code> / <code>write</code> 결과를 구분해야 한다	OSTEP 물리 463~469쪽	일치
worker 종료에는 stop 표시·깨우기 <code>join</code> 이 필요하다	C++ PDF 물리 814~817쪽(책 809~812쪽)	일치
callback 후 카메라 버퍼의 유효 기간과 전용 해제 함수	세 PDF에 특정 SDK 계약 없음	제조사 문서 확인이 필요한 조건부 항목으로 표시

재대조 결과, 특정 카메라 SDK의 동작을 PDF 근거인 것처럼 단정하지 않고 세 가지 소유권 모델로 분리했다. 또한 종료 순서에서 acquisition 중지와 camera 객체 파괴를 구분했다.

참고한 PDF

- [ostep.pdf](#) : event-based concurrency, AIO request buffer, file descriptor, file I/O, write buffering, DMA
- [ocs.pdf](#) : socket endpoint, interrupt-driven I/O, DMA, kernel I/O subsystem
- [cpp.pdf](#) : blocking 작업의 thread 분리, future를 통한 결과 전달, worker stop-notify-join

title: 비동기 작업 취소와 UI 정지 없는 이미지 처리 aliases:

tags:

created: 2026-08-20 updated: 2026-08-20

- 비동기 작업과 취소
- 스레드 안전한 UI 이미지 처리
- UI 프리징 방지
- CS
- OS
- Cpp
- Async
- Cancellation
- ThreadSafety
- UI
- ImageProcessing

비동기 작업 취소와 UI 정지 없는 이미지 처리

[!summary] 한 문장 정리

UI thread는 event 처리와 화면 반영만 담당하고, 이미지 buffer를 소유한 작업을 worker에게 넘긴 뒤, 취소는 협력적으로 요청하며, 결과는 generation을 재검사해 UI thread에서만 반영한다.

0. 문서 범위와 기존 문서 연결

이 문서는 다음 네 주제를 하나의 이미지 처리 pipeline으로 연결한다.

1. 비동기 작업
2. 작업 취소
3. 스레드 안전성
4. UI 정지 없는 이미지 처리

다음 일반 개념은 기존 문서를 선행 자료로 사용한다.

- 비동기·동시성·병렬성 구분: 02.동시성과 병렬성
- mutex-condition variable-종료 가능한 queue: 03.Mutex와 Condition Variable
- 이미지 buffer 소유권과 SDK 수명: 05.파일·소켓·비동기 IO와 Buffer
- interface-executor-단위 테스트: 06.C++ 객체지향 설계와 테스트 가능한 시스템

이 문서는 기존 설명을 반복하기보다 다음 실무 문제에 집중한다.

- 왜 이미지 처리를 worker로 옮겼는데도 UI가 멈추는가?
- 취소 버튼을 누르면 queued 작업과 running 작업은 각각 어떻게 되는가?
- 이전 이미지의 늦은 결과가 최신 화면을 덮지 않게 하려면 어떻게 하는가?
- worker가 UI widget을 직접 만지지 않게 하려면 어떻게 하는가?
- 창을 닫을 때 callback-worker-buffer를 어떤 순서로 정리하는가?

0.1 PDF 대조 범위

자료	직접 확인한 내용	PDF 물리 페이지
cpp.pdf	race condition, mutex, condition variable	744~770쪽 부근
cpp.pdf	promise/future, 예외 전달, <code>std::async</code>	796~810쪽 부근
cpp.pdf	작업 queue, stop flag, notify, worker join	812~817쪽 부근
ostep.pdf	thread의 주소 공간 공유와 독립 stack	287~310쪽 부근
ostep.pdf	condition variable과 상태 predicate	348~362쪽 부근
ostep.pdf	blocking/non-blocking, event loop, AIO request-completion 상태	397~404쪽 부근
ocs.pdf	thread와 병행성, 공유 데이터의 race, mutex-semaphore	스레드 장과 프로세스 동기화 장

[!note] UI와 작업 취소의 근거 범위

PDF는 특정 GUI framework의 UI-thread API나 C++20 `stop_token` 을 직접 설명하지 않는다. 따라서 UI dispatcher, latest-wins, cooperative cancellation 예제는 PDF에서 확인한 event loop·공유 메모리·condition variable·작업 queue 원칙을 조합한 설계 종합이다.

1. UI가 멈추는 이유

1.1 UI는 event loop 위에서 동작한다

대부분의 desktop UI framework는 하나의 UI thread에서 다음 작업을 반복한다.

TEXT

```
event queue에서 event 꺼내기
-> mouse/keyboard/window message 처리
-> layout·paint·animation 처리
-> 다음 event 처리
```

하나의 handler가 오래 반환하지 않으면 뒤의 event들이 처리되지 못한다.

TEXT

UI thread

[클릭 handler: 이미지 처리 2초 -----]

[paint][mouse][timer]

사용자 관찰: 창 이동·버튼·progress 표시가 2초 동안 멈춤

OSTEP의 event-driven server 설명에서 blocking system call 하나가 event loop 전체를 막는 것과 같은 구조다. UI event handler의 긴 CPU 계산, blocking I/O, `future.get()`, `thread.join()` 도 같은 결과를 만든다.

1.2 UI handler에서 피해야 할 작업

- 큰 이미지 decode-resize-filter-검사
- 동기 파일-network-camera I/O
- worker 완료를 기다리는 `future.get()`
- thread 종료를 기다리는 `join()`
- 긴 mutex 대기
- queue가 찰 때까지 기다리는 blocking push
- 대량 결과를 한 번에 widget에 삽입

[!important] worker를 만들었다고 자동으로 UI가 유동적이지는 않다

UI thread가 worker를 시작한 직후 `get()` 또는 `join()` 하면 결국 UI가 완료를 기다린다. 작업 시작과 결과 소비를 분리하고, 결과를 UI event queue에 다시 게시해야 한다.

2. 비동기 작업의 정확한 모델

2.1 비동기는 “나중에 완료를 받는 것”이다

TEXT

UI thread

Worker

submit(OwnedImage)

즉시 event loop로 복귀

process

post(Result)

UI queue에서 result 적용

호출 직후 UI thread가 다시 event loop로 돌아가야 한다. worker는 완료 결과나 오류를 callback, future, completion queue 중 하나로 전달한다.

2.2 한 작업이 가져야 하는 상태

CPP

```
enum class JobState {
    queued,
    running,
    cancel_requested,
    completed,
    cancelled,
    failed
};
```

실제 구현이 enum 하나를 반드시 가져야 하는 것은 아니지만 다음 사실은 구분해야 한다.

- queue에만 있고 아직 시작하지 않은가?
- worker가 buffer를 읽고 있는가?
- 취소 요청을 worker가 관찰했는가?
- 성공·실패·취소 중 어떤 결과로 종결되었는가?
- 결과가 만들어졌지만 UI에 아직 반영되지 않았는가?

2.3 요청·실행·전달은 서로 다른 단계다

TEXT

```
Submit
-> Queue
-> Start
-> Process
-> Produce result
-> Post to UI queue
-> Validate generation
-> Commit to UI
```

작업이 **completed** 여도 UI에 적용되기 전일 수 있다. 이 사이에 더 최신 요청이 들어오면 완료된 예전 결과도 폐기해야 할 수 있다.

3. 작업 취소

3.1 취소는 강제 종료가 아니다

일반적인 C++ 작업 취소는 cooperative cancellation이다.

FORMULA · CALCULATION

취소 요청자: `cancel_requested = true`

작업 수행자: 안전한 중단 지점에서 flag 확인

작업 수행자: 임시 결과 폐기·자원 정리 후 `cancelled`로 종료

thread를 임의의 instruction에서 강제로 멈추면 다음 상태가 남을 수 있다.

- mutex를 획득한 채 종료
- container 불변식이 깨진 중간 갱신
- SDK frame 미반납
- 파일-transaction 부분 반영
- callback 대상이 불완전한 상태

따라서 취소는 작업 코드가 일관성을 유지할 수 있는 지점에서 관찰한다.

3.2 queued 취소와 running 취소

상태	취소 방식	비용
queued	queue에서 제거하거나 cancelled 표시	거의 즉시
running	작업이 cancellation checkpoint에서 확인	checkpoint 간격만큼 지연
OS/SDK I/O 대기	API의 cancel 요청 후 completion 확인	API 계약에 의존
UI queue에 결과 게시 완료	callback에서 generation/alive 재검사 후 폐기	callback 자체는 실행될 수 있음

`cancel()` 함수가 반환한 시점과 작업이 buffer 접근을 끝낸 시점은 같지 않을 수 있다. 05.파일-소켓-비동기 IO와 Buffer에서 다룬 것처럼 OS/SDK 비동기 I/O는 취소 완료까지 확인해야 buffer를 해제할 수 있다.

3.3 cancellation checkpoint의 크기

CPP

```
for (std::size_t row = 0; row < height; ++row) {
    if (cancel_requested.load(std::memory_order_relaxed)) {
        return Cancelled{};
    }
    process_one_row(row);
}
```

매 pixel마다 atomic을 읽으면 반응은 빠르지만 overhead가 늘 수 있다. 이미지 처리에서는 row, tile, 단계 사이를 checkpoint로 두는 것이 이해하기 쉽다.

TEXT

```
decode -> [check] -> normalize -> [check] -> inference -> [check] -> annotate
```

한 단계가 3초 동안 interruption을 허용하지 않으면 취소 응답 시간도 최대 약 3초다. 취소 latency 요구를 먼저 정하고 작업 단위를 나눈다.

3.4 취소 결과는 실패와 다르다

CPP

```
enum class ResultKind {
    success,
    cancelled,
    failed
};
```

사용자가 새 이미지를 선택해 이전 작업을 취소한 것은 일반적으로 오류가 아니다. error log를 쌓거나 경고 dialog를 띄우기보다 정상적인 제어 흐름으로 처리한다.

3.5 취소 후 결과 경쟁

다음 순서가 가능하다.

TEXT

```
Worker: 계산 완료
UI:      취소 요청 또는 새 이미지 선택
Worker: 이전 결과를 UI queue에 post
UI:      이전 callback 실행
```

취소 flag만으로 이 경쟁을 완전히 막을 수 없다. UI callback이 실행되는 마지막 순간에 generation과 controller 생존 상태를 다시 확인해야 한다.

4. latest-wins와 모든 작업 보존

4.1 latest-wins가 적합한 경우

- slider를 움직일 때 preview 재계산
- 사용자가 파일을 연속 선택
- camera preview에서 최신 frame만 중요
- 검색어 입력에 따른 thumbnail 갱신

TEXT

```
Job 10 running
Job 11 arrives -> Job 10 cancel requested
Job 12 arrives -> queued Job 11 폐기, Job 12만 유지
```

queue가 무한히 늘지 않고 오래된 결과가 화면을 덮지 않는다.

4.2 모든 작업을 보존해야 하는 경우

- 검사 결과가 법적·품질 기록이 되는 생산 frame
- 업로드·저장처럼 각 요청의 완료가 필요
- batch 처리에서 모든 입력이 결과를 가져야 함

이 경우 latest-wins로 frame을 조용히 버리면 안 된다. bounded queue, backpressure, 명시적 reject, retry 정책이 필요하다.

요구	권장 정책
화면 preview	latest-only 또는 drop-oldest
모든 검사 기록	bounded FIFO + 입력 제어
사용자 단일 명령	이전 명령 취소 후 최신 유지 가능
파일 저장	작업별 성공·실패 결과 보존

5. 스레드 안전성

5.1 thread-safe의 범위를 말해야 한다

“이 클래스는 thread-safe다”만으로는 부족하다.

- 어떤 public 함수들을 동시에 호출할 수 있는가?
- callback은 어느 thread에서 오는가?
- 결과 객체는 immutable인가?
- `stop()` 과 `submit()` 이 경쟁하면 어떤 결과인가?
- UI 객체는 어느 thread에서만 접근할 수 있는가?

5.2 UI 객체는 UI thread에 가둔다

worker가 계산을 마쳤다고 widget을 직접 변경하면 안 된다. UI framework의 dispatcher/event queue에 결과 적용 함수를 게시한다.

TEXT

Worker thread

- > immutable Result 생성
- > `ui_dispatcher.post(...)`

UI thread

- > generation 확인
- > widget/model 변경

이 방식은 UI 상태에 mutex를 추가하는 것보다 ownership이 명확하다. UI 상태는 UI thread 하나가 소유하고, worker는 값 결과만 전달한다.

5.3 공유 상태를 최소화한다

공유하기 쉬운 데이터 구조보다 이동 가능한 값 객체를 우선한다.

CPP

```
struct OwnedImage {
    std::uint64_t id{};
    std::size_t width{};
    std::size_t height{};
    std::vector<unsigned char> pixels;
};
```

UI가 `OwnedImage` 를 worker queue로 이동하면 UI와 worker가 같은 pixel vector를 동시에 수정하지 않는다.

5.4 mutex와 atomic의 역할을 구분한다

- 여러 필드의 관계와 queue 불변식: mutex
- 독립적인 단일 flag-generation: atomic 후보
- 기다림: condition variable
- 소유권 이전: 값 이동 또는 `unique_ptr`

CPP

```
{
    std::lock_guard lock(mutex_);
    pending_ = std::move(job);
    stopping_ = false;
}
ready_.notify_one();
```

atomic 여러 개를 조합해 복잡한 상태 머신을 만들면 일관된 snapshot을 얻기 어렵다. queue-running token-shutdown 상태처럼 함께 변해야 하는 값은 한 mutex 아래 둔다.

5.5 condition variable은 상태 predicate와 함께 사용한다

CPP

```
ready_.wait(lock, [this] {
    return stopping_ || pending_.has_value();
});
```

notification은 상태 자체가 아니다. spurious wakeup이나 이미 소비된 event를 고려해 깨어난 뒤 predicate를 다시 검사한다.

5.6 오래 잡은 lock도 UI를 멈춘다

UI thread가 짧은 mutex를 획득하려고 했는데 worker가 그 lock을 잡고 이미지 처리 전체를 수행하면 UI는 멈춘다.

CPP

```
// 나쁜 구조
std::lock_guard lock(mutex_);
result_ = process_large_image(image_); // lock을 너무 오래 보유
```

CPP

```
// 좋은 구조
OwnedImage local;
{
    std::lock_guard lock(mutex_);
    local = std::move(pending_image_);
}
auto result = process_large_image(local); // 공유 상태 밖에서 계산
```

6. 이미지 buffer 수명

6.1 주소만 넘기는 것은 비동기 전달이 아니다

CPP

```
void on_open_clicked() {
    std::vector<unsigned char> pixels = load();
    worker.post([&pixels] { process(pixels); });
} // pixels 소멸 가능
```

lambda가 reference로 capture한 지역 buffer는 worker 실행 전에 소멸할 수 있다.

CPP

```
worker.post([pixels = std::move(pixels)]() mutable {
    process(pixels);
});
```

작업이 값을 소유하게 만들면 완료 취소까지 buffer 수명이 작업 수명과 같아진다.

6.2 camera SDK buffer

SDK callback pointer는 callback 이후 재사용될 수 있다. UI 이미지 작업으로 넘길 때는 다음 중 하나가 필요하다.

- callback 안에서 app-owned `OwnedImage` 로 복사
- 공식 SDK retain/lock handle을 RAII lease로 보유
- app-owned buffer pool의 상태를 Filling/Ready/Processing으로 관리

pointer나 `std::span` 만 queue에 넣어서는 수명이 연장되지 않는다.

6.3 결과 이미지도 소유권이 필요하다

UI callback이 실행될 때까지 결과 pixel memory가 살아 있어야 한다. 결과 값을 callback lambda로 이동 capture하면 dispatcher queue가 그 수명을 보유한다.

CPP

```
ui.post([result = std::move(result)]() mutable {
    apply_to_ui(std::move(result));
});
```

7. UI 정지 없는 pipeline

7.1 권장 흐름

TEXT

UI event

- > 입력 검증
- > image ownership 확보
- > generation 증가
- > 이전 token cancel
- > latest job 교체
- > worker notify
- > 즉시 return

Worker

- > latest job 인수
- > tile/row마다 cancel 확인
- > result 생성
- > UI dispatcher에 callback 게시

UI callback

- > controller alive 확인
- > generation == latest 확인
- > widget/model 갱신

7.2 generation이 필요한 이유

TEXT

generation 1: 큰 이미지, 먼저 시작했지만 늦게 완료
 generation 2: 작은 이미지, 나중에 시작했지만 먼저 완료

완료 순서만 믿으면 generation 1이 마지막에 화면을 덮는다. submit 시 증가한 generation을 결과에 넣고 UI commit 직전에 최신 값과 비교한다.

CPP

```
if (result.generation != latest_generation.load()) {
    return; // stale result
}
```

7.3 progress update를 너무 자주 보내지 않는다

worker가 pixel마다 UI event를 post하면 UI queue가 progress event로 가득 찬다. 다음 중 하나를 사용한다.

- 50~100 ms에 한 번만 게시
- 마지막 게시보다 일정 비율 이상 변했을 때만 게시
- atomic progress 하나를 갱신하고 UI timer가 주기적으로 읽음
- queue에서 이전 progress event를 최신 값으로 병합

8. C++20 통합 예제: latest-only image controller

다음 예제는 특정 GUI framework 대신 thread-safe `IUiDispatcher` interface를 사용한다. 실제 Qt-Win32-wxWidgets adapter는 해당 framework의 UI queue 게시 API로 `post()` 를 구현하면 된다.

CPP · 1/8

```
#include <atomic>
#include <cassert>
#include <chrono>
#include <condition_variable>
#include <cstddef>
#include <cstdint>
#include <deque>
#include <functional>
#include <memory>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <thread>
#include <utility>
#include <vector>

struct OwnedImage {
    std::uint64_t id{};
    std::size_t width{};
    std::size_t height{};
    std::vector<unsigned char> pixels;
};

struct ProcessedImage {
    std::uint64_t generation{};
    std::uint64_t source_id{};
    std::vector<unsigned char> pixels;
};

class IUiDispatcher {
public:
    virtual ~IUiDispatcher() = default;
    virtual void post(std::function<void()> callback) = 0;
};

class IImageView {
public:
    virtual ~IImageView() = default;
    virtual void show(ProcessedImage image) = 0; // UI thread only
};

class ImageController {
private:
    struct CancelFlag {
        std::atomic_bool requested{false};
    };
};
```

CPP · 2/8

```

struct Job {
    std::uint64_t generation{};
    OwnedImage image;
    std::shared_ptr<CancelFlag> cancel;
};

struct DeliveryState {
    std::atomic_bool alive{true};
    std::atomic_uint64_t latest_generation{0};
    IImageView* view{}; // UI thread에서만 사용
};

public:
    ImageController(IUiDispatcher& ui, IImageView& view)
        : ui_(ui),
          delivery_(std::make_shared<DeliveryState>()),
          worker_([this] { worker_loop(); }) {
        delivery_>view = &view;
    }

    ~ImageController() {
        stop();
    }

    ImageController(const ImageController&) = delete;
    ImageController& operator=(const ImageController&) = delete;

    // UI thread에서 빠르게 호출하고 반환한다.
    void submit(OwnedImage image) {
        if (image.width == 0 ||
            image.height == 0 ||
            image.pixels.size() % image.width != 0 ||
            image.pixels.size() / image.width != image.height) {
            throw std::invalid_argument("invalid image dimensions");
        }

        auto cancel = std::make_shared<CancelFlag>();
        std::uint64_t generation{};

        {
            std::lock_guard lock(mutex_);
            if (stopping_) {
                throw std::runtime_error("controller is stopping");
            }
        }
    }

```

CPP · 3/8

```

        generation =
            delivery_->latest_generation.fetch_add(1) + 1;

        if (pending_) {
            pending_>cancel->requested.store(true);
        }
        if (running_cancel_) {
            running_cancel_>requested.store(true);
        }

        pending_ = Job{
            generation,
            std::move(image),
            std::move(cancel)
        };
    }
    ready_.notify_one();
}

// 요청 중인 결과를 화면에 반영하지 않는다.
void cancel_current() noexcept {
    delivery_>latest_generation.fetch_add(1);
    {
        std::lock_guard lock(mutex_);
        if (pending_) {
            pending_>cancel->requested.store(true);
            pending_.reset();
        }
        if (running_cancel_) {
            running_cancel_>requested.store(true);
        }
    }
}

// 예제 계약: UI thread에서 한 번 호출하며 worker 종료까지 기다린다.
void stop() noexcept {
    delivery_>alive.store(false);
    {
        std::lock_guard lock(mutex_);
        if (stopping_) {
            return;
        }
    }
}

```

CPP · 4/8

```

        stopping_ = true;
        if (pending_) {
            pending_>cancel->requested.store(true);
            pending_.reset();
        }
        if (running_cancel_) {
            running_cancel_>requested.store(true);
        }
    }

    ready_.notify_all();
    if (worker_.joinable()) {
        worker_.join();
    }
}

private:
    static std::optional<ProcessedImage> process(const Job& job) {
        ProcessedImage output;
        output.generation = job.generation;
        output.source_id = job.image.id;
        output.pixels.resize(job.image.pixels.size());

        const std::size_t row_size = job.image.width;
        for (std::size_t row = 0; row < job.image.height; ++row) {
            if (job.cancel->requested.load(std::memory_order_relaxed)) {
                return std::nullopt;
            }

            const std::size_t begin = row * row_size;
            const std::size_t end = begin + row_size;
            for (std::size_t index = begin; index < end; ++index) {
                output.pixels[index] =
                    static_cast<unsigned char>(255 - job.image.pixels[index]);
            }
        }
        return output;
    }

    void worker_loop() noexcept {
        for (;;) {
            std::optional<Job> job;
            {
                std::unique_lock lock(mutex_);
                ready_.wait(lock, [this] {
                    return stopping_ || pending_.has_value();
                });
            }

```

CPP · 5/8

```

        if (stopping_) {
            return;
        }

        job = std::move(pending_);
        pending_.reset();
        running_cancel_ = job->cancel;
    }

    auto result = process(*job); // lock 밖에서 계산

    {
        std::lock_guard lock(mutex_);
        if (running_cancel_ == job->cancel) {
            running_cancel_.reset();
        }
    }

    if (!result || job->cancel->requested.load()) {
        continue;
    }

    auto delivery = delivery_;
    ui_.post([delivery, image = std::move(*result)]() mutable {
        // 이 callback은 UI thread에서 실행된다는 계약이다.
        if (!delivery->alive.load()) {
            return;
        }
        if (image.generation != delivery->latest_generation.load()) {
            return; // 늦게 도착한 stale result
        }
        delivery->view->show(std::move(image));
    });
}

}

IUiDispatcher& ui_;
std::shared_ptr<DeliveryState> delivery_;
std::mutex mutex_;
std::condition_variable ready_;
std::optional<Job> pending_; // latest-only slot
std::shared_ptr<CancelFlag> running_cancel_;
bool stopping_ = false;
std::thread worker_;
};

```

CPP · 6/8

```
// 실제 GUI event queue를 대신하는 검증용 dispatcher다.
class TestUiDispatcher final : public IUiDispatcher {
public:
    void post(std::function<void()> callback) override {
        {
            std::lock_guard lock(mutex_);
            callbacks_.push_back(std::move(callback));
        }
        ready_.notify_one();
    }

    template <class Predicate>
    bool run_until(Predicate done, std::chrono::milliseconds timeout) {
        const auto deadline = std::chrono::steady_clock::now() + timeout;
        while (!done()) {
            std::function<void()> callback;
            {
                std::unique_lock lock(mutex_);
                if (!ready_.wait_until(lock, deadline, [this] {
                    return !callbacks_.empty();
                })) {
                    return done();
                }
                callback = std::move(callbacks_.front());
                callbacks_.pop_front();
            }
            callback(); // 테스트를 실행한 main(UI) thread에서 수행
        }
        return true;
    }

    void drain() {
        for (;;) {
            std::function<void()> callback;
            {
                std::lock_guard lock(mutex_);
                if (callbacks_.empty()) {
                    return;
                }
                callback = std::move(callbacks_.front());
                callbacks_.pop_front();
            }
        }
    }
};
```

CPP · 7/8

```

        callback();
    }
}

private:
    std::mutex mutex_;
    std::condition_variable ready_;
    std::deque<std::function<void()>> callbacks_;
};

class RecordingView final : public IImageView {
public:
    RecordingView() : ui_thread_(std::this_thread::get_id()) {}

    void show(ProcessedImage image) override {
        assert(std::this_thread::get_id() == ui_thread_);
        shown_.push_back(std::move(image));
    }

    std::size_t count() const { return shown_.size(); }
    std::uint64_t last_source_id() const { return shown_.back().source_id; }

private:
    std::thread::id ui_thread_;
    std::vector<ProcessedImage> shown_;
};

OwnedImage make_image(
    std::uint64_t id,
    std::size_t width,
    std::size_t height,
    unsigned char value) {
    return OwnedImage{
        id,
        width,
        height,
        std::vector<unsigned char>(width * height, value)
    };
}

int main() {
    using namespace std::chrono_literals;

```

CPP · 8/8

```

TestUiDispatcher ui;
RecordingView view;
ImageController controller{ui, view};

// 첫 결과가 이미 UI queue에 들어왔더라도 generation 검사가 폐기한다.
controller.submit(make_image(1, 1024, 1024, 10));
controller.submit(make_image(2, 32, 32, 20));

const bool completed = ui.run_until(
    [&view] { return view.count() == 1; },
    2s);

assert(completed);
assert(view.last_source_id() == 2); // 최신 요청만 화면에 표시

controller.stop();
ui.drain(); // 남은 callback은 alive 검사로 안전하게 폐기
}

```

8.1 예제의 thread 계약

함수·객체	호출 thread	동시 호출 계약
submit()	UI thread	예제에서는 UI 직렬 호출을 기본으로 함
cancel_current()	UI thread	running token에 atomic 취소 요청
worker_loop()	전용 worker	UI 객체 직접 접근 금지
IUiDispatcher::post()	worker 가능	구현이 thread-safe해야 함
IImageView::show()	UI thread만	dispatcher가 thread affinity 보장
stop()	UI thread, 종료 시 1회	worker join까지 기다림

submit() 과 stop() 을 여러 외부 thread에서 동시에 호출해야 한다면 별도의 API 계약과 lifecycle 동기화가 추가로 필요하다. UI controller를 UI thread에 가두면 이 복잡성을 크게 줄일 수 있다.

8.2 DeliveryState 가 필요한 이유

worker가 UI callback을 게시한 뒤 controller가 먼저 파괴될 수 있다. callback이 this 를 capture하면 use-after-free가 된다.

예제는 callback이 독립된 shared_ptr<DeliveryState> 만 보유하게 한다.

- controller 종료 시 alive = false
- 게시된 callback은 state 수명을 유지
- UI에서 callback 실행 시 alive 가 false면 view 접근 없이 종료
- UI callback과 controller 파괴가 같은 UI thread에서 직렬화된다는 계약 사용

8.3 왜 요청 교체 때 join() 하지 않는가?

새 이미지를 선택할 때 이전 worker를 join하면 UI thread가 취소 checkpoint까지 기다려 다시 멈춘다. 요청 교체는 flag만 설정하고 즉시 반환한다. join() 은 controller 최종 종료에서만 수행한다.

9. 실제 GUI framework에 연결할 때

구현해야 할 핵심은 하나다.

CPP

```
class FrameworkUiDispatcher final : public IUiDispatcher {
public:
    void post(std::function<void()> callback) override {
        // framework의 thread-safe UI event 게시 API로 callback을 넘긴다.
    }
};
```

확인할 framework 계약:

- worker thread에서 게시 API를 호출해도 되는가?
- callback은 반드시 UI thread에서 실행되는가?
- window가 파괴되면 대기 callback은 폐기되는가?
- receiver가 파괴되었을 때 callback capture는 어떻게 되는가?
- application shutdown 중 새 event 게시가 허용되는가?

특정 framework의 객체 수명 기능을 쓰더라도 generation 검사는 별도로 필요하다. 객체가 살아 있다는 것과 그 결과가 최신이라는 것은 다른 문제다.

10. 오류·완료·취소 전달

10.1 결과를 tagged state로 전달하기

CPP

```
struct JobResult {
    enum class Kind { success, cancelled, failed } kind;
    std::uint64_t generation{};
    std::optional<ProcessedImage> image;
    std::string error;
};
```

다음 불변식을 정할 수 있다.

- success면 **image** 가 존재하고 **error** 는 비어 있음
- cancelled면 UI 오류 dialog를 띄우지 않음
- failed면 error가 존재하고 최신 generation일 때만 화면에 표시

10.2 worker 예외 경계

worker thread 함수 밖으로 예외가 빠져나가지 않게 한다.

CPP

```
try {
    auto result = process(job);
    post_success(std::move(result));
} catch (const std::exception& error) {
    post_failure(job.generation, error.what());
} catch (...) {
    post_failure(job.generation, "unknown error");
}
```

C++ PDF의 `packaged_task` ·future 설명처럼 비동기 결과는 값뿐 아니라 예외도 전달할 수 있다. UI application에서는 future를 UI thread에서 blocking `get()` 하기보다 completion callback이 오류 결과를 UI queue에 게시하는 구조가 자연스럽다.

11. 종료 순서

11.1 window 닫기

FORMULA · CALCULATION

새 submit 차단

- > delivery alive=false
- > pending job 폐기
- > running job cancel 요청
- > worker 깨우기
- > worker가 checkpoint에서 종료
- > join
- > UI dispatcher의 남은 callback은 alive 검사로 폐기
- > view/window 파괴

view를 먼저 파괴하면 이미 게시된 callback이 dangling view에 접근할 수 있다.

11.2 종료 중 UI freeze

예제의 `stop()` 은 `join()` 하므로 running 단계가 취소 flag를 오래 확인하지 않으면 창 닫기가 늦어진다.

해결 방법:

- 긴 알고리즘을 tile-stage로 나누고 checkpoint를 추가
- 취소 가능한 SDK API 사용
- 종료 화면을 먼저 숨기고 별도 lifecycle coordinator가 완료를 기다림
- timeout 뒤 process 자체 종료가 허용되는지 제품 정책으로 결정

강제 thread 종료는 일반적인 해결책이 아니다.

12. 성능과 응답성

12.1 응답성 지표

- UI event 처리 지연
- submit 함수 실행 시간
- cancel 요청부터 worker 종료까지 시간
- queue 대기 시간
- 이미지 처리 시간
- result 생성부터 UI commit까지 시간
- stale result 폐기 수
- frame drop 수

평균만 보면 긴 꼬리 지연을 놓친다. p95/p99 latency도 관찰한다.

12.2 복사와 zero-copy

UI 정지를 막기 위해 무조건 zero-copy가 필요한 것은 아니다.

- callback에서 짧게 복사하고 빠르게 반환하면 SDK buffer를 빨리 돌려줄 수 있음
- 복사 없는 frame lease는 처리 중 SDK pool을 점유
- 큰 이미지는 buffer pool로 할당 비용을 줄일 수 있음
- UI 표시용 축소 image와 정밀 검사 원본을 분리할 수 있음

정확성과 lifetime이 먼저이며, 복사 비용은 측정 후 최적화한다.

12.3 worker 수

worker를 늘린다고 항상 빨라지지 않는다.

- memory bandwidth 경쟁
- cache miss와 false sharing
- 결과 reorder 증가
- UI callback 폭주
- 외부 library가 thread-safe하지 않을 수 있음

preview latest-wins는 worker 하나로도 충분한 경우가 많다. 모든 frame 처리 pipeline은 bounded pool과 순서 정책이 필요하다.

13. 테스트 전략

13.1 단위 테스트

- generation 1 결과가 generation 2 이후 도착하면 폐기되는가?
- pending job을 새 job이 교체하는가?
- running 취소 요청을 checkpoint가 관찰하는가?
- cancelled 결과가 오류로 표시되지 않는가?
- controller 파괴 뒤 게시 callback이 view를 호출하지 않는가?
- worker 예외가 UI failure 결과로 변환되는가?

13.2 동시성 component test

- submit과 worker 인수가 반복 경쟁해도 data race가 없는가?
- stop 시 condition variable에서 worker가 반드시 깨어나는가?
- 결과 callback이 UI thread에서만 실행되는가?
- 큰 이미지 취소 latency가 요구 범위 안인가?
- queue:buffer 사용량에 상한이 있는가?

13.3 timeout은 성공 조건이 아니라 안전장치다

예제의 2초 timeout은 "2초 기다리면 작업이 끝날 것"이라는 test 방식이 아니다. 실제 성공 조건은 `view.count() == 1` 이고 timeout은 deadlock 시 test process가 영원히 멈추지 않게 하는 상한이다.

14. 흔한 실패 패턴

14.1 UI thread에서 `future.get()`

CPP

```
auto future = std::async(std::launch::async, process, image);
auto result = future.get(); // UI thread block
show(result);
```

계산은 worker에서 해도 UI는 기다리므로 멈춘다.

14.2 detached thread가 UI 객체 capture

CPP

```
std::thread([this] {
    auto result = process();
    widget_>setImage(result);
}).detach();
```

객체 수명, UI thread affinity, 종료 대기를 모두 잃는다.

14.3 취소 flag만 설정하고 buffer 해제

worker가 아직 buffer를 읽고 있을 수 있다. 취소 완료 또는 worker join 전까지 소유 buffer를 유지한다.

14.4 오래된 결과가 최신 화면을 덮음

취소 flag를 마지막에 한 번만 확인하거나 generation이 없으면 발생한다. UI commit 직전에 최신 generation을 재검사한다.

14.5 unbounded image queue

입력 속도가 처리 속도보다 빠르면 메모리와 지연이 계속 증가한다. preview는 latest-only, 기록 작업은 bounded queue와 backpressure를 사용한다.

14.6 worker가 mutex를 잡고 UI callback 호출

UI callback이 controller를 다시 호출하면 deadlock이 가능하다. 공유 상태를 정리하고 lock을 해제한 뒤 dispatcher에 게시한다.

15. 실무 체크리스트

비동기 작업

- ☐ UI handler가 작업을 제출한 뒤 즉시 반환하는가?
- ☐ UI thread에서 `get()` · `join()` · blocking I/O를 하지 않는가?
- ☐ 작업이 input buffer를 완료까지 소유하는가?
- ☐ 성공·실패·취소 상태를 구분하는가?

작업 취소

- ☐ queued와 running 취소를 구분하는가?
- ☐ cancellation checkpoint 간 최대 시간이 정해져 있는가?
- ☐ 취소 요청과 취소 완료를 구분하는가?
- ☐ stale result를 UI commit 직전에 다시 거르는가?

스레드 안전성

- ☐ UI 객체는 UI thread에서만 접근하는가?
- ☐ queue·pending·running·stopping 상태가 같은 mutex로 보호되는가?
- ☐ condition variable이 predicate와 함께 사용되는가?
- ☐ 긴 계산을 lock 밖에서 수행하는가?
- ☐ callback이 controller의 raw `this` 수명에 의존하지 않는가?

이미지 처리와 종료

- ☐ preview와 모든 frame 보존 중 어느 정책인지 명시했는가?
- ☐ SDK callback buffer의 소유권을 확인했는가?
- ☐ progress event를 제한하는가?
- ☐ stop -> cancel -> notify -> join -> view destroy 순서인가?
- ☐ 취소 latency와 UI p95/p99 지연을 측정하는가?

16. 핵심 결론

1. UI 정지는 event loop의 handler가 오래 반환하지 않을 때 발생한다.
2. 비동기 처리는 worker 생성이 아니라 submit과 UI commit을 분리하는 구조다.
3. 작업 취소는 강제 종료가 아니라 안전한 checkpoint에서 관찰하는 협력적 protocol이다.
4. 취소 flag만으로 늦은 결과를 막을 수 없으므로 generation 검사가 필요하다.
5. UI 객체는 UI thread가 소유하고 worker는 값 결과만 dispatcher로 전달한다.
6. 이미지 buffer는 작업이 완료·취소 완료될 때까지 명시적으로 소유해야 한다.
7. preview는 latest-wins가 적합하지만 모든 검사 기록에는 bounded FIFO가 필요하다.
8. 요청 교체 때 join하지 않고 최종 종료에서 cancel-notify-join한다.

17. 작성 후 검증 결과

17.1 Markdown·C++ 검증

- UTF-8 replacement character가 없는지 확인했다.
- frontmatter, 제목 계층과 모든 code fence의 짝을 확인했다.
- 02.동시성과 병렬성, 03.Mutex와 Condition Variable, 05.파일·소켓·비동기 IO와 Buffer, 06.C++ 객체지향 설계와 테스트 가능한 시스템의 내부 링크가 실제 파일을 가리키는지 확인했다.
- 요청한 비동기 작업·작업 취소·스레드 안전성·UI 정지 없는 이미지 처리가 각각 독립된 절과 통합 예제에 포함되었는지 확인했다.
- 통합 예제를 MSVC C++20 `/W4 /permissive-` 로 컴파일했다.
- 최신 요청이 이전 요청을 대체하는 경쟁을 포함해 실행 파일을 200회 반복 실행했다.

17.2 PDF 2차 재대조

문서의 주장	다시 확인한 PDF 근거	결과
event loop에서 blocking call을 실행하면 다른 event 처리가 멈춘다	ostep.pdf 물리 398~400쪽	일치
event/AIO 방식은 요청-buffer-완료 뒤 continuation 상태를 따로 보관한다	ostep.pdf 물리 400~403쪽	일치
thread는 주소 공간을 공유하고 각자 stack을 가진다	ostep.pdf 물리 287쪽, ocs.pdf 스레드 장	일치
thread 완료를 기다릴 때 join을 사용하며 stack 지역 주소를 결과로 반환하면 위험하다	ostep.pdf 물리 304~306쪽	일치
condition variable은 predicate를 while로 재검사해야 한다	ostep.pdf 물리 309쪽, 348쪽 부근	일치
공유 상태에는 race가 발생할 수 있고 mutex로 임계 영역을 보호한다	cpp.pdf 물리 744~759쪽, ocs.pdf 동기화 장	일치
future의 <code>get()</code> 은 결과 준비를 기다리므로 UI thread에서 호출하면 block될 수 있다	cpp.pdf 물리 799~800쪽	일치
<code>std::async</code> 의 <code>async/deferred</code> 정책은 실행 시점을 다르게 한다	cpp.pdf 물리 807~808쪽	일치
작업 queue는 mutex/CV로 보호하고 종료 시 stop-notify-join한다	cpp.pdf 물리 812~817쪽	일치
cooperative cancellation-generation-UI dispatcher의 구체 패턴	세 PDF에 직접 구현 없음	PDF 원칙에서 도출한 설계 종합으로 명시

2차 대조 결과, 특정 GUI framework의 thread affinity와 callback 폐기 동작을 일반 PDF가 보장하는 것처럼 단정하지 않았다. framework adapter를 구현할 때 별도 문서에서 확인해야 할 계약으로 분리했다.

참고한 PDF

- [cpp.pdf](#) : C++ thread, race condition, mutex/CV, promise/future, async와 thread pool 종료
- [ostep.pdf](#) : thread 공유 메모리, condition variable, event loop의 blocking 문제, AIO 상태 관리
- [ocs.pdf](#) : thread와 병행성, 공유 데이터 동기화, mutex와 semaphore

다중 카메라 이미지 검사 스레드 설계와 멈춤 진단

[!summary]

카메라 수신, 이미지 검사, UI 갱신은 서로 요구하는 지연 시간과 스레드 규칙이 다르다. 세 단계를 한 스레드나 하나의 공유 버퍼에 묶으면 UI 정지, 프레임 손상, 간헐적 멈춤, 종료 교착이 발생하기 쉽다. 안전한 기본 구조는 **수신 콜백은 짧게, 프레임은 명시적으로 소유, 검사는 bounded queue 뒤의 worker에서, UI 갱신은 UI dispatcher로, 취소는 협력적 protocol로** 처리하는 것이다.

0. 문서 범위와 기존 문서와의 관계

이 문서는 다음 다섯 질문에 집중한다.

1. 이미지 수신 스레드와 UI 스레드를 왜 분리해야 하는가?
2. 검사 중 UI가 멈추는 직접 원인은 무엇인가?
3. 여러 카메라가 같은 데이터에 접근하면 어떤 문제가 생기는가?
4. 프로그램이 가끔만 멈추고 재현되지 않는 이유는 무엇인가?
5. 실행 중인 검사 작업을 어떻게 안전하게 취소하는가?

일반 개념은 다음 문서에 이미 정리되어 있다.

- 01.프로세스와 스레드: 스레드, 스케줄링, 교착 상태
- 02.동시성과 병렬성: data race, happens-before, 병렬 처리
- 03.Mutex와 Condition Variable: mutex, 조건 변수, 생산자-소비자
- 05.파일:소켓:비동기 IO와 Buffer: 비동기 I/O와 카메라 버퍼 수명
- 07.비동기 작업 취소와 UI 이미지 처리: 비동기 작업, generation, UI dispatcher

따라서 이 문서에서는 개념을 반복하기보다 **다중 카메라 검사 프로그램에서 실제로 어떻게 조합하고 진단하는지**를 다룬다.

[!warning] PDF 근거의 범위

세 PDF는 스레드, 동기화, 생산자-소비자, 교착 상태, 이벤트 기반 처리의 원리를 설명한다. 특정 카메라 제조사의 callback 재진입, buffer retain, acquisition stop, frame requeue 계약은 정의하지 않는다. SDK 관련 수명과 해제 규칙은 반드시 해당 제조사 문서에서 별도로 확인해야 한다.

1. 권장 전체 구조

MERMAID

flowchart LR

```

C1["Camera 1 callback"] --> O1["Owned Frame 복사 또는 SDK retain"]
C2["Camera 2 callback"] --> O2["Owned Frame 복사 또는 SDK retain"]
C3["Camera N callback"] --> O3["Owned Frame 복사 또는 SDK retain"]
O1 --> Q["Bounded inspection queue"]
O2 --> Q
O3 --> Q
Q --> W1["Inspection worker 1"]
Q --> W2["Inspection worker 2"]
W1 --> R["Immutable Result"]
W2 --> R
R --> D["UI dispatcher"]
D --> U["UI thread: 화면 상태 변경"]
X["Cancel / shutdown state"] --> Q
X --> W1
X --> W2
X --> D

```

각 경계의 의미는 다음과 같다.

경계	입력 측 책임	출력 측 책임
SDK callback → owned frame	callback이 유효한 동안 복사 또는 retain	작업 완료까지 buffer 수명 유지
owned frame → queue	queue가 받을 수 있는지 확인	FIFO, latest-only, drop 정책 적용
queue → worker	대기 중 취소-종료를 predicate에 포함	긴 계산 중 취소 checkpoint 확인

경계	입력 측 책임	출력 측 책임
worker → result	결과를 불변 값으로 완성	부분 결과를 UI와 동시에 수정하지 않음
result → UI dispatcher	UI 객체를 직접 건드리지 않음	UI thread에서 최신 generation 확인 후 반영

이 구조에서 “스레드 분리”는 단순히 `std::thread` 를 많이 만드는 것이 아니다. 소유권, 대기, 취소, 전달 방향을 경계마다 정의하는 것이다.

2. 이미지 수신 스레드와 UI 스레드를 분리해야 하는 이유

2.1 UI thread는 이벤트 루프를 계속 돌려야 한다

대부분의 GUI framework는 하나의 UI thread가 다음 작업을 순서대로 처리한다.

- 마우스와 키보드 입력
- 창 이동·크기 변경
- paint와 animation
- button callback
- timer와 UI message
- 화면 객체의 생성·수정·파괴

UI handler가 카메라 수신이나 이미지 검사에서 오래 머물면 다음 event를 처리하지 못한다. 운영체제가 창에 “응답 없음”을 표시하는 현상은 계산이 틀려서가 아니라 **UI event queue가 제때 소비되지 않기 때문**이다.

C++

```
void MainWindow::onInspectClicked() {
    auto frame = camera.wait_frame(); // blocking I/O
    auto result = inspect(frame);      // 긴 CPU 계산
    resultLabel.setText(result.text);  // 여기까지 UI event loop가 정지
}
```

계산을 worker에 넣었더라도 UI에서 즉시 기다리면 동일하게 멈춘다.

C++

```
void MainWindow::onInspectClicked() {
    auto future = std::async(std::launch::async, inspect, current_frame);
    auto result = future.get(); // UI thread가 완료까지 대기하므로 여전히 정지
    show(result);
}
```

핵심은 “다른 스레드를 만들었는가”가 아니라 **UI thread가 기다리지 않고 event loop로 복귀하는가**다.

2.2 카메라 callback은 빠르게 반환해야 한다

카메라 SDK의 callback thread는 보통 다음 frame 수신, driver buffer 반환, 내부 acquisition pipeline과 연결된다. callback 안에서 검사나 UI 호출을 오래 수행하면:

- 다음 frame 전달이 지연된다.
- SDK 내부 buffer pool이 고갈될 수 있다.
- frame drop 또는 acquisition timeout이 발생할 수 있다.
- SDK가 내부 lock을 보유한 채 callback을 호출한다면 lock 역전 위험이 커진다.
- UI thread와 동기 호출을 주고받다가 교착 상태가 생길 수 있다.

callback에서 해야 할 일은 보통 다음으로 제한한다.

1. 최소 metadata를 읽는다.
2. SDK buffer 수명 계약에 따라 애플리케이션 소유 메모리로 복사하거나 retain한다.
3. bounded queue에 넣거나 명시적 drop 정책을 적용한다.
4. 즉시 반환한다.

2.3 두 스레드는 요구하는 우선순위가 다르다

작업	중요한 목표	오래 걸릴 때의 대응
UI	짧은 응답 지연, 공정한 event 처리	작업을 worker로 넘김
수신	frame 획득 주기와 buffer 반환 기한	callback을 짧게 유지
검사	처리량과 정확도	worker 수, batch, SIMD/GPU 조정
저장	데이터 보존과 순서	bounded FIFO와 backpressure 사용

하나의 스레드에 묶으면 가장 느린 단계가 전체 pipeline을 멈춘다. 분리하면 각 단계의 병목과 실패를 독립적으로 측정할 수 있다.

2.4 UI 객체는 UI thread 전용인 경우가 많다

worker가 label, bitmap, window handle을 직접 수정하는 것은 framework의 thread-affinity 계약을 위반할 수 있다. 문제는 즉시 crash로 나타나지 않을 수도 있다. 내부 상태가 우연히 맞을 때는 동작하고, paint-resize-destroy와 겹칠 때만 실패할 수 있다.

C++

```
// worker thread에서 직접 호출하면 위험하다.
imageView.setImageBitmap(result.bitmap);

// framework가 제공하는 post/invoke/dispatcher를 이용한다.
uiDispatcher.post([result = std::move(result)]() mutable {
    imageView.setImageBitmap(std::move(result.bitmap));
});
```

`post()` 는 비동기 전달이어야 한다. worker가 UI callback의 완료를 기다리는 동기 `invoke()` 를 사용하면 UI가 worker를 `join()` 하거나 같은 mutex를 기다릴 때 교착 상태가 생길 수 있다.

3. 검사 중 UI가 멈추는 원인

UI 정지는 한 가지 원인만 있는 증상이 아니다. CPU가 높은지, 낮은지, 특정 mutex에서 기다리는지부터 구분해야 한다.

3.1 UI thread에서 긴 계산 수행

가장 직접적인 원인이다.

- convolution, threshold, labeling
- 큰 이미지 format 변환
- CPU와 GPU 사이의 동기 복사
- 압축·해제
- 수천 개 결과를 한 번에 widget에 추가

해결은 계산을 worker로 옮기고 UI에는 표시 가능한 최종 snapshot만 전달하는 것이다.

3.2 UI thread의 blocking wait

다음 호출은 UI thread에서 특히 위험하다.

- `thread.join()`
- `future.get()` 또는 `wait()`
- predicate 없는 `condition_variable::wait()`
- timeout 없는 SDK `wait_frame()`
- 동기 파일·네트워크 I/O
- semaphore나 mutex의 무기한 대기

worker 사용 자체가 UI 비동기를 보장하지 않는다. UI가 worker의 완료를 기다리면 실행 위치만 분리되었을 뿐 의존 관계는 동기다.

3.3 worker가 UI에 필요한 mutex를 오래 보유

CPP

```
std::lock_guard lock(model_mutex);
inspect_large_image(shared_model); // lock을 잡은 채 수백 ms 계산
```

UI가 같은 mutex로 진행률이나 결과를 읽으면 화면이 멈춘다. 공유 상태는 lock 안에서 snapshot으로 복사하고, 계산은 lock 밖에서 한다.

CPP

```
ModelSnapshot snapshot;
{
    std::lock_guard lock(model_mutex);
    snapshot = make_snapshot(shared_model);
}
auto result = inspect_large_image(snapshot); // lock 밖
```

3.4 worker → UI 동기 호출과 UI → worker 대기의 순환

다음 관계는 전형적인 종료 교착이다.

TEXT

UI thread: worker.join() 완료를 기다림
 worker thread: UI thread의 synchronous invoke 완료를 기다림

두 스레드 모두 상대가 진행해야 깨어나므로 영원히 멈춘다. worker → UI는 비동기 **post**, UI 종료는 먼저 callback 생성을 차단한 뒤 별도 종료 절차로 수행한다.

3.5 UI message flood

worker가 frame마다 여러 진행률 event를 보내면 UI queue가 입력 속도를 따라가지 못한다. UI thread가 살아 있어도 클릭과 paint event가 수천 개 progress event 뒤에 밀려 “멈춘 것처럼” 보인다.

대응 방법:

- 진행률은 30~100 ms 간격으로 제한한다.
- 같은 종류 update는 마지막 값만 남긴다.
- preview는 latest-only로 처리한다.
- table/log widget에는 batch로 추가한다.
- queue depth를 metric으로 기록한다.

3.6 무한 queue와 메모리 압력

카메라 입력률이 검사 처리율보다 빠르는데 모든 frame을 무한히 쌓으면:

- queue latency가 계속 증가한다.
- 대형 buffer allocation과 해제가 반복된다.
- working set이 커져 page fault와 cache miss가 증가한다.
- UI paint와 검사 worker가 memory bandwidth를 경쟁한다.
- 결국 전체 시스템이 멈춘 것처럼 느껴질 수 있다.

queue는 반드시 용량이 있어야 하며, 용도별 overflow 정책을 결정해야 한다.

용도	권장 정책
실시간 preview	최신 frame 유지, 오래된 frame drop
모든 제품 검사 기록	bounded FIFO, 상위 장비에 backpressure 또는 명시적 reject
통계 표시	일정 구간 aggregate
디버그 화면	sampling 또는 rate limit

3.7 종료 handler에서 모든 thread를 즉시 join

창 닫기 handler가 무한 I/O나 긴 검사 중인 thread를 **join()** 하면 종료 화면이 멈춘다. 먼저 입력을 차단하고, I/O를 깨우고, 취소를 요청해야 한다. **join()** 은 thread가 종료 조건을 관찰할 수 있게 만든 뒤에 수행한다.

4. 여러 카메라가 같은 데이터에 접근할 때 생기는 문제

4.1 data race와 undefined behavior

서로 다른 두 thread가 같은 메모리에 접근하고, 하나 이상이 쓰며, happens-before 관계가 없으면 C++ data race다. 결과가 가끔 틀리는 정도가 아니라 undefined behavior다.

```
CPP
struct SharedResult {
    int camera_id;
    std::vector<std::uint8_t> pixels;
};

SharedResult latest;

// camera 1 thread
latest.camera_id = 1;
latest.pixels.assign(cam1_data, cam1_data + size);

// camera 2 thread: 동시에 같은 객체를 변경하면 data race
latest.camera_id = 2;
latest.pixels.assign(cam2_data, cam2_data + size);
```

`std::vector` 는 여러 thread가 같은 객체의 크기와 storage를 동시에 변경하도록 보장하지 않는다. reallocation과 읽기가 겹치면 iterator-pointer 무효화와 메모리 손상까지 이어질 수 있다.

4.2 data race가 없어도 논리적 race가 남을 수 있다

각 필드 접근에 mutex를 붙였더라도 하나의 논리적 snapshot을 여러 임계 영역으로 나누면 camera ID와 pixel이 서로 다른 frame에서 올 수 있다.

```
CPP
set_camera_id(1);          // lock/unlock
set_pixels(cam1_frame);    // lock/unlock 사이에 camera 2가 끼어들 수 있음
```

하나의 결과를 완성한 뒤 한 번에 교체한다.

```
CPP
Frame complete{camera_id, frame_id, owned_pixels};
{
    std::lock_guard lock(latest_mutex);
    latest = std::move(complete);
}
```

더 좋은 방법은 mutable “현재 작업 중 frame”을 공유하지 않고, queue가 완성된 값 객체의 소유권을 이동하도록 만드는 것이다.

4.3 같은 SDK buffer의 재사용

callback이 받은 raw pointer를 그대로 queue에 저장하면 callback 반환 뒤 SDK가 그 buffer를 다음 frame에 재사용할 수 있다.

TEXT

```
camera callback: pointer P를 frame 100으로 전달
callback return
SDK:             P에 frame 101을 덮어씀
inspection:      P를 frame 100이라고 믿고 읽음
```

결과는 찢어진 이미지, camera 간 frame 혼합, 간헐적 검사 오판, use-after-free 형태로 나타난다.

안전한 선택은 둘 중 하나다.

1. callback 안에서 애플리케이션 소유 buffer로 복사한다.
2. SDK의 공식 retain/lock/requeue 기능으로 buffer를 보유하고 전용 deleter를 RAII에 넣는다.

SDK 계약을 모르는 상태에서 raw pointer를 비동기 worker에 넘기는 것은 안전하다고 판단할 수 없다.

4.4 전역 설정의 동시 변경

검사 중 threshold, ROI, calibration을 UI가 수정하면 한 frame 안에서도 서로 다른 설정이 섞일 수 있다. 설정은 작업 시작 시 immutable snapshot으로 만든다.

CPP

```
auto config = std::make_shared<const InspectionConfig>(ui_config);
queue.push(Job{frame, config});
```

작업은 전달받은 `const` snapshot만 읽고, UI의 이후 변경은 다음 작업부터 적용한다.

4.5 ABBA 교착 상태

카메라별 mutex와 공용 결과 mutex를 반대 순서로 얻으면 교착 상태가 가능하다.

TEXT

```
camera 1 thread: camera1_mutex 획득 → result_mutex 대기
UI thread:      result_mutex 획득 → camera1_mutex 대기
```

대응:

- 여러 lock이 필요한 구조 자체를 줄인다.
- 반드시 전역 lock 순서를 문서화한다.
- 함께 필요한 lock은 `std::scoped_lock(m1, m2)` 로 획득한다.
- 외부 SDK 함수나 UI callback을 lock 안에서 호출하지 않는다.
- lock을 보유한 채 `join`, `wait`, blocking I/O를 호출하지 않는다.

4.6 contention과 false sharing

정확성 문제가 없어도 모든 camera thread가 하나의 mutex, allocator, log sink, 결과 map을 경쟁하면 지연이 급증한다. 카메라별 독립 상태를 분리하고, 공용 aggregation은 완성된 결과만 받도록 한다.

자주 갱신되는 카메라별 counter가 같은 cache line에 붙어 있으면 false sharing이 생길 수 있다. 먼저 profiler로 확인하고 필요할 때 `alignas(std::hardware_destructive_interference_size)` 같은 분리를 검토한다. 정렬은 정확성 해결책이 아니라 성능 조정이다.

4.7 권장 데이터 소유 구조

```

CPP

struct Frame {
    int camera_id{};
    std::uint64_t frame_id{};
    std::uint64_t generation{};
    std::chrono::steady_clock::time_point received_at;
    std::vector<std::uint8_t> pixels; // 이 frame이 독점 소유
};

struct InspectionResult {
    int camera_id{};
    std::uint64_t frame_id{};
    std::uint64_t generation{};
    bool pass{};
    double score{};
};

```

- 수신 callback은 `Frame` 을 완성한다.
- queue로 `std::move(frame)` 한다.
- worker는 자신의 frame만 읽는다.
- 결과는 별도의 값으로 완성한다.
- UI는 결과를 받은 뒤 화면 모델을 갱신한다.

공유 mutable buffer를 여러 단계가 돌려 쓰는 것보다 메모리 사용량은 명시적으로 보이지만, 정확성과 수명 검증은 훨씬 쉬워진다.

5. 프로그램이 가끔 멈추지만 재현되지 않는 이유

5.1 스케줄링 순서는 실행마다 달라진다

OS scheduler는 thread 실행 시점과 선점을 매번 동일하게 보장하지 않는다. 다음 요소가 작은 timing 차이를 만든다.

- CPU 부하와 core 배치
- camera frame 도착 간격
- cache hit/miss와 page fault
- 디버거와 log 출력
- compiler 최적화
- background process와 interrupt
- 네트워크·디스크·USB 지연

오류가 특정 수 마이크로초의 interleaving에서만 나타나면 수백 번 정상 실행한 뒤 한 번 멈출 수 있다.

5.2 디버깅을 시작하면 사라지는 Heisenbug

log, breakpoint, debugger는 실행 순서를 바꾼다.

- `printf` 자체가 내부 lock과 I/O를 사용한다.
- breakpoint는 다른 thread가 먼저 끝날 시간을 준다.
- Debug build는 최적화와 메모리 배치가 다르다.
- 관찰 코드가 race window를 넓히거나 줄인다.

따라서 “debugger에서 재현되지 않는다”는 동시성 문제가 아니라는 증거가 아니다.

5.3 멈춤 증상의 분류

분류	의미	CPU 경향	대표 원인
deadlock	서로 필요한 자원을 기다림	낮음	ABBA lock, UI↔worker 동기 대기
lost wake-up/잘못된 predicate	상태는 변했지만 waiter가 진행하지 못함	낮음	상태와 CV를 다른 규칙으로 보호
무한 blocking I/O	외부 event가 오지 않아 대기	낮음	timeout 없는 SDK wait
livelock	서로 양보·재시도하며 상태만 바꿈	높을 수 있음	<code>try_lock</code> 재시도 protocol
starvation	특정 thread가 자원을 계속 못 얻음	혼합	lock 경쟁, 불공정 queue
busy loop	종료 조건을 관찰하지 못하고 반복	높음	atomic/predicate 오류
data race/UB	메모리 모델 위반으로 결과 불명	다양	shared vector, non-atomic stop flag
overload	입력률이 처리율보다 높음	높음	unbounded queue, UI message flood

5.4 condition variable 사용 오류

조건 변수는 event 저장소가 아니다. 공유 상태가 진실이고 notification은 다시 확인하라는 신호다.

CPP

```
std::unique_lock lock(mutex);
cv.wait(lock, [&] {
    return !queue.empty() || stopping;
});
```

predicate를 `while` 의미로 재검사해야 하는 이유:

- spurious wake-up이 가능하다.
- notify 뒤 mutex를 다시 얻기 전에 다른 thread가 상태를 바꿀 수 있다.
- 여러 waiter 중 다른 waiter가 먼저 데이터를 소비할 수 있다.
- 종료도 정상적인 wake 조건에 포함해야 한다.

상태는 mutex 안에서 바꾸고, waiter가 같은 mutex와 같은 predicate로 확인해야 한다.

5.5 종료 flag의 data race

```

CPP
bool stop = false;

// UI thread
stop = true;

// worker thread
while (!stop) { /* ... */ } // 동기화가 없으면 data race

```

`volatile` 로 바꾸어도 thread 동기화가 되지 않는다. 단일 독립 flag라면 `std::atomic_bool`, queue 상태와 함께 불변식을 구성한다면 같은 mutex로 보호한다. C++20에서는 `std::jthread` 와 `std::stop_token` 도 사용할 수 있다.

5.6 callback 재진입과 객체 파괴 경쟁

종료 중 camera 객체를 파괴하는데 SDK callback이 이미 진입했거나 곧 호출될 수 있다면 callback의 `this`, UI 객체, buffer pool이 먼저 사라질 수 있다. 정상 운전에서는 드물고 창을 닫는 순간에만 나타나므로 재현이 어렵다.

종료 protocol은 다음을 명시해야 한다.

- 새 callback 진입을 SDK 수준에서 어떻게 막는가?
- 이미 실행 중인 callback이 반환했음을 어떻게 확인하는가?
- callback이 참조하는 context는 언제까지 살아 있는가?
- SDK buffer가 모두 반납되었음을 어떻게 확인하는가?

5.7 재현보다 먼저 관측 가능성을 만든다

모든 event에 다음 correlation 정보를 기록한다.

```

TEXT
monotonic_time
thread_id
camera_id
frame_id
generation
event_name
queue_depth
wait_duration_us
processing_duration_us
result_state

```

권장 event:

- `callback_enter` , `callback_exit`
- `frame_owned` , `enqueue_ok` , `enqueue_drop`
- `dequeue` , `inspect_begin` , `cancel_seen` , `inspect_end`
- `ui_post` , `ui_commit` , `stale_drop`
- `stop_acquisition_begin/end`
- `queue_close` , `worker_exit` , `join_begin/end`

wall clock은 시간이 보정될 수 있으므로 구간 측정에는 `std::chrono::steady_clock` 을 사용한다.

[!tip]

멈춘 순간의 "마지막 log 한 줄"만으로 원인을 확정하지 않는다. 그 thread가 다음에 어떤 mutex, I/O, condition variable, UI 호출을 기다렸는지 wait chain과 thread stack을 함께 확보해야 한다.

6. 검사 작업을 안전하게 취소하는 방법

6.1 취소 요청과 취소 완료는 다르다

`cancel_requested = true` 는 worker에게 요청을 전달했을 뿐이다. worker가 그 상태를 관찰하고, 부분 작업을 버리고, buffer와 임시 자원을 정리해 작업 loop를 빠져나와야 취소가 완료된다.

TEXT

Running → CancelRequested → cleanup → Cancelled

다음과 같이 상태를 구분하면 종료와 UI 처리가 명확해진다.

TEXT

Queued | Running | Succeeded | Failed | Cancelled

6.2 강제 thread 종료가 위험한 이유

임의 지점에서 thread를 죽이면 다음 상태가 남을 수 있다.

- mutex를 획득한 채 종료
- SDK buffer 미반납
- 파일·GPU·카메라 handle 누수
- 결과 객체가 반만 갱신된 상태
- allocator 또는 library 내부 불변식 손상

안전한 기본값은 **협력적 취소**다. 작업이 안전한 checkpoint에서 중지 요청을 확인한다.

6.3 취소 checkpoint 위치

CPP

```
for (std::size_t y = 0; y < height; ++y) {
    if (stop.stop_requested() || generation != latest_generation.load()) {
        return Cancelled{};
    }
    process_row(y);
}
```

checkpoint 간격은 취소 지연과 검사 성능의 절충이다. 한 단계가 2초 걸리는데 단계 끝에서만 확인하면 최악의 취소 지연도 약 2초다. 요구되는 취소 latency를 정한 뒤 tile-row-batch 경계에 배치한다.

6.4 queued 작업과 running 작업을 따로 처리

- queued: queue에서 제거하거나 취소된 generation으로 남겨 worker가 즉시 폐기한다.
- running: token/generation을 checkpoint에서 관찰한다.
- completed but not displayed: UI commit 직전 generation을 다시 확인한다.
- blocking SDK I/O: SDK 취소 API, timeout, handle close 등 별도 해제 수단이 필요하다.

취소 flag만으로는 이미 UI queue에 들어간 오래된 결과를 막지 못한다. 결과에 generation을 넣고 UI 적용 직전에 최신 generation과 비교한다.

6.5 frame 소유권은 취소 완료까지 유지한다

취소 요청 직후 입력 buffer를 해제하면 worker가 아직 읽는 중일 수 있다. frame을 작업 값이 소유하게 하고 worker가 반환할 때 RAII로 해제되게 한다.

CPP

```
struct Job {
    std::uint64_t generation;
    std::shared_ptr<const Frame> frame;
};
```

`shared_ptr` 가 무조건 정답이라는 뜻은 아니다. queue에서 worker 하나로 소유권이 이동한다면 `unique_ptr<Frame>` 이나 값 이동이 더 명확하다. 여러 비동기 단계가 같은 immutable frame을 실제로 공유할 때만 `shared_ptr<const Frame>` 을 사용한다.

6.6 취소 중 새 작업 정책

업무 의미에 따라 정한다.

시나리오	정책
화면 preview	이전 generation 취소, 최신 frame만 유지
제품별 필수 검사	임의 drop 금지, bounded FIFO와 설비 backpressure
사용자가 검사 recipe 변경	이전 generation 결과 폐기, 새 config snapshot 사용
프로그램 종료	새 입력 금지, source 중지, running 취소 또는 drain

6.7 안전한 종료 순서

일반적인 순서는 다음과 같다.

1. `accepting = false` : 새 검사 요청을 받지 않는다.
2. 카메라 acquisition을 중지해 새 callback 생성을 막는다.
3. 실행 중 callback이 모두 반환했는지 SDK 계약으로 확인한다.
4. 최신 generation을 증가시키고 queued 작업을 폐기하거나 drain 정책을 적용한다.
5. 검사 worker에 stop을 요청한다.
6. condition variable과 blocking I/O를 깨운다.
7. 수신 thread와 검사 worker를 `join()` 한다.
8. UI에 대기 중인 stale callback이 결과를 반영하지 못하게 delivery state를 무효화한다.
9. frame/buffer를 반환하고 카메라-GPU-파일 자원을 파괴한다.
10. 마지막으로 관련 UI 객체를 파괴한다.

“검사 thread를 카메라보다 먼저 없앨 것인가?”라는 질문은 **카메라 객체 파괴**와 **acquisition 중지**를 구분해야 한다.

- 새 frame 생산은 먼저 중지한다.
- 생산자가 더 이상 queue에 넣지 않음을 확인한다.
- 그 다음 소비자인 검사 worker를 drain하거나 취소해 종료한다.
- SDK buffer를 worker가 참조한다면 모든 worker가 반납한 뒤 camera handle/buffer pool을 파괴한다.

7. C++20 통합 예제

다음 예제는 framework와 SDK에 독립적인 핵심 구조다.

- camera callback은 SDK buffer를 즉시 소유 복사한다.
- bounded queue가 과부하를 제한한다.
- 여러 worker는 immutable frame을 처리한다.
- generation으로 실행 중 작업과 늦게 도착한 UI 결과를 취소한다.
- UI 객체 대신 thread-safe dispatcher에 callback을 전달한다.
- 종료는 새 입력 차단 → queue close → stop request → join 순서다.

CPP · 1/8

```

#include <algorithm>
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <cstdint>
#include <cstdint>
#include <deque>
#include <functional>
#include <iostream>
#include <memory>
#include <mutex>
#include <optional>
#include <span>
#include <stop_token>
#include <thread>
#include <utility>
#include <vector>

using namespace std::chrono_literals;

struct Frame {
    int camera_id{};
    std::uint64_t frame_id{};
    std::uint64_t generation{};
    std::chrono::steady_clock::time_point received_at;
    std::vector<std::uint8_t> pixels;
};

struct InspectionResult {
    int camera_id{};
    std::uint64_t frame_id{};
    std::uint64_t generation{};
    std::uint64_t checksum{};
};

template <class T>
class BoundedQueue {
public:
    explicit BoundedQueue(std::size_t capacity) : capacity_(capacity) {}

    bool try_push(T value) {
        {
            std::lock_guard lock(mutex_);
            if (closed_ || queue_.size() >= capacity_) {
                return false;
            }
        }
    }

```

CPP · 2/8

```

        queue_.push_back(std::move(value));
    }
    ready_.notify_one();
    return true;
}

std::optional<T> pop(std::stop_token stop) {
    std::unique_lock lock(mutex_);
    const bool awakened = ready_.wait(lock, stop, [this] {
        return closed_ || !queue_.empty();
    });

    if (!awakened || queue_.empty()) {
        return std::nullopt;
    }

    T value = std::move(queue_.front());
    queue_.pop_front();
    return value;
}

void discard_pending() {
    std::lock_guard lock(mutex_);
    queue_.clear();
}

void close(bool discard) {
    {
        std::lock_guard lock(mutex_);
        closed_ = true;
        if (discard) {
            queue_.clear();
        }
    }
    ready_.notify_all();
}

private:
    const std::size_t capacity_;
    std::mutex mutex_;
    std::condition_variable_any ready_;
    std::deque<T> queue_;
    bool closed_{false};
};

```

CPP · 3/8

```

class UiDispatcher {
public:
    virtual ~UiDispatcher() = default;
    virtual void post(std::function<void()> callback) = 0;
};

struct DeliveryState {
    std::atomic_bool alive{true};
    std::atomic_uint64_t latest_generation{1};
};

class InspectionPipeline {
public:
    using ResultHandler = std::function<void(InspectionResult)>;

    InspectionPipeline(
        UiDispatcher& ui,
        ResultHandler on_result,
        std::size_t worker_count,
        std::size_t queue_capacity)
        : ui_(ui),
          on_result_(std::move(on_result)),
          queue_(queue_capacity),
          delivery_(std::make_shared<DeliveryState>()) {
        workers_.reserve(worker_count);
        for (std::size_t i = 0; i < worker_count; ++i) {
            workers_.emplace_back([this](std::stop_token stop) {
                worker_loop(stop);
            });
        }
    }

    InspectionPipeline(const InspectionPipeline&) = delete;
    InspectionPipeline& operator=(const InspectionPipeline&) = delete;

    ~InspectionPipeline() {
        stop();
    }
}

```

CPP · 4/8

```

// SDK callback thread에서 호출한다.
bool on_sdk_frame(
    int camera_id,
    std::uint64_t frame_id,
    std::span<const std::uint8_t> sdk_buffer) {
    if (!accepting_.load(std::memory_order_acquire)) {
        return false;
    }

    Frame frame;
    frame.camera_id = camera_id;
    frame.frame_id = frame_id;
    frame.generation =
        delivery->latest_generation.load(std::memory_order_acquire);
    frame.received_at = std::chrono::steady_clock::now();

    // callback 반환 뒤에도 유효하도록 애플리케이션이 소유한다.
    frame.pixels.assign(sdk_buffer.begin(), sdk_buffer.end());

    // cancel과 경쟁해 stale frame이 들어가도 worker가 generation으로 폐기한다.
    return queue_.try_push(std::move(frame));
}

// UI thread에서 호출해도 대기하지 않고 즉시 반환한다.
void cancel_current() {
    delivery->latest_generation.fetch_add(1, std::memory_order_acq_rel);
    queue_.discard_pending();
}

// 실제 시스템에서는 이 함수를 부르기 전에 SDK acquisition을 중지하고
// 실행 중 callback이 모두 반환했음을 확인해야 한다.
void stop() {
    bool expected = false;
    if (!stopped_.compare_exchange_strong(expected, true)) {
        return;
    }

    accepting_.store(false, std::memory_order_release);
    delivery->latest_generation.fetch_add(1, std::memory_order_acq_rel);
    delivery->alive.store(false, std::memory_order_release);
    queue_.close(true);

    for (auto& worker : workers_) {
        worker.request_stop();
    }
}

```

CPP · 5/8

```

    for (auto& worker : workers_) {
        if (worker.joinable()) {
            worker.join();
        }
    }
}

private:
    std::optional<InspectionResult> inspect(
        const Frame& frame,
        std::stop_token stop) const {
        std::uint64_t checksum = 0;

        constexpr std::size_t batch = 256;
        for (std::size_t begin = 0; begin < frame.pixels.size(); begin += batch) {
            if (stop.stop_requested() ||
                frame.generation !=
                    delivery_>->latest_generation.load(std::memory_order_acquire)) {
                return std::nullopt;
            }

            const auto end = (std::min)(begin + batch, frame.pixels.size());
            for (std::size_t i = begin; i < end; ++i) {
                checksum += frame.pixels[i];
            }
        }

        return InspectionResult{
            frame.camera_id,
            frame.frame_id,
            frame.generation,
            checksum};
    }

    void worker_loop(std::stop_token stop) {
        while (!stop.stop_requested()) {
            auto frame = queue_.pop(stop);
            if (!frame) {
                return;
            }

            auto result = inspect(*frame, stop);
            if (!result) {
                continue;
            }
        }
    }

```

CPP · 6/8

```

    auto delivery = delivery_;
    auto handler = on_result_;
    ui_.post([delivery,
              handler = std::move(handler),
              result = std::move(*result)]() mutable {
        if (!delivery->alive.load(std::memory_order_acquire)) {
            return;
        }
        if (result.generation !=
            delivery->latest_generation.load(std::memory_order_acquire)) {
            return;
        }
        handler(std::move(result)); // UI thread에서 실행
    });
}

UiDispatcher& ui_;
ResultHandler on_result_;
BoundedQueue<Frame> queue_;
std::shared_ptr<DeliveryState> delivery_;
std::vector<std::jthread> workers_;
std::atomic_bool accepting_{true};
std::atomic_bool stopped_{false};
};

```

// 실제 GUI의 post/invokeLater에 해당하는 작은 test dispatcher다.

```

class TestUiDispatcher final : public UiDispatcher {
public:
    void post(std::function<void()> callback) override {
        std::lock_guard lock(mutex_);
        callbacks_.push_back(std::move(callback));
    }

    void drain_on_ui_thread() {
        std::deque<std::function<void()>> local;
        {
            std::lock_guard lock(mutex_);
            local.swap(callbacks_);
        }
        for (auto& callback : local) {
            callback();
        }
    }
};

```

CPP · 7/8

```

    }

    std::size_t pending_count() const {
        std::lock_guard lock(mutex_);
        return callbacks_.size();
    }

private:
    mutable std::mutex mutex_;
    std::deque<std::function<void()>> callbacks_;
};

int main() {
    TestUiDispatcher ui;
    std::atomic_int committed{0};

    InspectionPipeline pipeline(
        ui,
        [&](InspectionResult result) {
            std::cout << "camera=" << result.camera_id
                << " frame=" << result.frame_id
                << " checksum=" << result.checksum << '\n';
            ++committed;
        },
        2,
        8);

    const std::vector<std::uint8_t> camera1(4096, 1);
    const std::vector<std::uint8_t> camera2(4096, 2);

    pipeline.on_sdk_frame(1, 100, camera1);
    pipeline.on_sdk_frame(2, 200, camera2);

    const auto first_deadline = std::chrono::steady_clock::now() + 2s;
    while (committed.load() < 2 &&
        std::chrono::steady_clock::now() < first_deadline) {
        ui.drain_on_ui_thread();
        std::this_thread::sleep_for(1ms);
    }
    if (committed.load() != 2) {
        return 1;
    }
}

```

CPP · 8/8

```
// 결과가 UI queue에 들어간 뒤 cancel해도 commit 직전 검사가 폐기한다.
pipeline.on_sdk_frame(1, 101, camera1);
const auto stale_deadline = std::chrono::steady_clock::now() + 2s;
while (ui.pending_count() == 0 &&
       std::chrono::steady_clock::now() < stale_deadline) {
    std::this_thread::sleep_for(1ms);
}
if (ui.pending_count() == 0) {
    return 2;
}
pipeline.cancel_current();
ui.drain_on_ui_thread();
if (committed.load() != 2) {
    return 3;
}

pipeline.stop();
ui.drain_on_ui_thread(); // stop 이전에 post된 stale callback도 폐기

return committed.load() == 2 ? 0 : 4;
}
```

7.1 예제에서 보장하는 것

위험	대응
SDK buffer 재사용	callback 안에서 <code>vector</code> 로 소유 복사
여러 camera의 shared buffer race	frame마다 독립된 값 객체
queue 과부하	<code>try_push</code> 와 고정 capacity
wait 중 종료 불가	<code>stop_token</code> , <code>close</code> , <code>notify_all</code>
실행 중 취소 지연	256 byte batch checkpoint
취소 후 늦은 결과 표시	UI commit 직전 generation 재검사
파괴된 controller 캡처	UI callback은 <code>DeliveryState</code> 만 공유
UI 직접 접근	dispatcher를 통해 UI thread에서 handler 실행

7.2 예제를 제품 코드로 옮길 때 추가할 것

- SDK 전용 acquisition stop과 callback 완료 barrier
- camera별 drop counter와 queue latency metric
- frame size-stride-pixel format 검증
- exception boundary와 실패 결과 전달
- 검사 설정의 immutable snapshot
- preview와 필수 검사 queue 분리
- 메모리 pool 또는 SDK retain 기반 zero-copy 검토
- framework 객체의 weak reference 또는 lifetime token
- shutdown timeout과 thread stack dump 수집

`TestUiDispatcher` 의 `sleep_for` 는 제품 동기화 방식이 아니다. 테스트 예제를 짧게 유지하기 위한 시간 여유일 뿐이며, 실제 테스트에서는 latch, promise, event, predicate를 사용해 완료 조건을 기다려야 한다.

8. 멈춤을 진단하는 실전 절차

8.1 1단계: 정지인지 과부하인지 구분

1. UI heartbeat가 계속 증가하는가?
2. process CPU 사용률은 높은가 낮은가?
3. queue depth가 계속 증가하는가?
4. camera callback rate와 inspection completion rate는 얼마인가?
5. UI message queue 또는 dispatcher backlog가 증가하는가?

CPU가 낮고 모든 counter가 멈췄다면 wait/deadlock 가능성이 높다. CPU가 높고 queue depth가 증가한다면 overload나 busy loop를 먼저 본다.

8.2 2단계: 멈춘 순간의 모든 thread stack 수집

한 thread만 보지 말고 다음 관계를 찾는다.

- UI가 어떤 lock, join, future, SDK call을 기다리는가?
- 해당 자원을 가진 worker는 무엇을 기다리는가?
- callback thread가 UI 또는 공용 mutex를 기다리는가?
- 모든 worker가 같은 CV에서 자는데 queue에는 데이터가 있는가?
- SDK 내부 wait에서 timeout 없이 멈춰 있는가?

wait-for graph를 그리면 순환 대기가 보인다.

TEXT

UI → worker join → UI synchronous invoke

또는:

TEXT

camera callback → result mutex → UI model mutex → camera stop

8.3 3단계: 시간과 queue를 수치로 확인

권장 metric:

- callback p50/p95/p99 duration
- enqueue 성공·drop 수
- queue depth와 oldest job age
- 검사 p50/p95/p99 duration

- cancellation latency
- UI post-to-commit latency
- lock wait/hold duration
- acquisition stop-to-last-callback duration

평균만 보면 드문 tail latency를 놓친다. “가끔 멈춤”은 p99 또는 최대값에서 드러나는 경우가 많다.

8.4 4단계: 동시성 검사 도구 사용

- ThreadSanitizer: data race 탐지에 유용
- AddressSanitizer: use-after-free, buffer overflow
- debugger wait chain/thread view: deadlock과 blocking 위치
- profiler/trace: CPU hot path, lock contention, queue latency
- 반복·부하·종료 stress test: timing window 확대

ThreadSanitizer가 조용하다고 논리적 race, 교착 상태, SDK 계약 위반이 없다는 뜻은 아니다. 각 도구가 찾는 오류 범위를 구분한다.

8.5 재현 가능성을 높이는 stress 조건

- 여러 camera를 동일 시점에 시작·중지
- frame rate를 처리량보다 높게 설정
- 검사 중 recipe를 빠르게 교체
- 창 닫기와 acquisition callback을 반복 경쟁
- queue capacity를 작게 설정
- worker 수를 1, 2, core 수 이상으로 변경
- cancellation checkpoint 직전·직후에 test hook 삽입
- SDK buffer pool 크기를 최소 허용값으로 낮춤

임의 `sleep` 만 추가하면 운 좋게 race를 숨길 수 있다. test hook, barrier, latch로 원하는 실행 순서를 강제로 구성하는 편이 낫다.

9. 반드시 테스트할 시나리오

9.1 수신과 소유권

- callback 반환 직후 SDK buffer 내용을 바꿔도 검사 frame은 변하지 않는가?
- camera 1과 camera 2가 동시에 callback해도 frame metadata가 섞이지 않는가?
- queue full일 때 drop/reject 정책이 명확하게 기록되는가?
- 잘못된 stride, size, format을 queue 전에 거부하는가?

9.2 UI responsiveness

- 최대 크기 이미지 처리 중 UI heartbeat p99가 요구 범위 안인가?
- worker가 결과를 초당 수천 건 post해도 coalescing이 동작하는가?
- UI thread에서 `join` , `future.get` , SDK wait가 호출되지 않는가?
- UI 객체 파괴 후 callback이 실행되어도 접근하지 않는가?

9.3 취소

- queued frame이 즉시 폐기되는가?
- running 검사가 정해진 latency 안에 checkpoint를 관찰하는가?
- 취소 직전 완료된 결과가 UI queue에 있어도 stale로 폐기되는가?
- 취소가 failure dialog나 error log 폭주로 처리되지 않는가?
- 취소 중 frame buffer가 너무 일찍 해제되지 않는가?

9.4 종료

- acquisition stop과 callback 진입이 경쟁해도 안전한가?
- worker가 CV, semaphore, SDK wait 중이어도 종료 가능한가?
- queue empty, full, 처리 중 각각에서 종료 가능한가?
- 여러 번 `stop()` 해도 안전한가?
- 종료 시 모든 callback, worker, UI post의 순서를 trace로 확인할 수 있는가?

9.5 반복 검증

- 동일 시나리오를 수천 회 반복한다.
- Debug와 최적화 Release를 모두 실행한다.
- worker 수와 camera 수 조합을 바꾼다.
- AddressSanitizer와 ThreadSanitizer 계열 검사를 별도 구성으로 실행한다.
- timeout은 성공 조건이 아니라 test process가 영원히 멈추는 것을 막는 상한으로 사용한다.

10. 자주 하는 잘못된 해결

10.1 모든 곳에 하나의 전역 mutex 사용

data race 일부는 사라질 수 있지만 UI, callback, worker가 하나의 lock을 경쟁한다. lock 안에서 검사나 SDK 호출을 하면 정지와 교착 가능성이 커진다. 데이터 소유권을 분리하고 임계 영역을 작게 만든다.

10.2 `volatile bool stop`

`volatile` 은 멀티스레드 원자성과 happens-before를 제공하지 않는다. `atomic` , mutex-protected state, `stop_token` 을 사용한다.

10.3 thread를 강제 종료

lock, buffer, handle, 부분 결과의 정리 시점을 잃는다. 취소 가능한 I/O와 협력적 checkpoint를 설계한다.

10.4 camera callback에서 UI를 동기 호출

SDK 내부 lock과 UI 종료가 결합될 수 있다. callback은 소유권 확보와 enqueue만 하고 반환한다.

10.5 raw SDK pointer를 queue에 저장

포인터를 받았다는 사실은 소유권을 뜻하지 않는다. 복사하거나 공식 retain 계약을 RAll로 감싼다.

10.6 queue를 무한히 키움

drop을 피한 것이 아니라 실패 시점을 메모리 고갈까지 미룬 것이다. 용량과 overflow 정책을 명시한다.

10.7 취소 flag만 확인하고 UI 결과는 무조건 반영

검사가 취소 요청과 거의 동시에 끝나 UI queue에 결과를 넣을 수 있다. UI commit 시 generation과 lifetime을 다시 검사한다.

10.8 로그가 없으니 deadlock이라고 단정

무한 I/O, lost notification, overload, memory corruption도 비슷하게 보인다. 모든 thread stack, queue metric, wait chain으로 분류한다.

11. 설계 점검표

스레드 경계

- ☐ camera callback은 검사와 UI 호출을 수행하지 않는가?
- ☐ UI thread는 blocking wait를 수행하지 않는가?
- ☐ worker → UI 전달은 비동기 post인가?
- ☐ UI 객체는 UI thread에서만 변경하는가?

데이터와 버퍼

- ☐ frame마다 camera ID, frame ID, generation이 함께 이동하는가?
- ☐ SDK buffer의 유효 기간과 반납 함수가 문서로 확인되었는가?
- ☐ callback 이후 사용할 frame은 소유 복사 또는 retain되는가?
- ☐ 여러 camera가 같은 mutable **vector** 나 bitmap을 공유하지 않는가?
- ☐ 검사 설정은 immutable snapshot인가?

동기화

- ☐ 모든 shared mutable state에 하나의 일관된 동기화 규칙이 있는가?
- ☐ 조건 변수 predicate에 queue data와 stop 상태가 함께 포함되는가?
- ☐ 여러 lock의 순서가 통일되어 있는가?
- ☐ lock 안에서 SDK, UI, join, blocking I/O를 호출하지 않는가?
- ☐ 긴 계산은 lock 밖에서 수행하는가?

부하 제어

- ☐ queue capacity가 제한되어 있는가?
- ☐ preview drop과 필수 검사 backpressure를 구분하는가?
- ☐ UI progress/result update가 rate limit 또는 coalesce되는가?
- ☐ queue depth와 oldest job age를 관측하는가?

취소와 종료

- ☐ 취소 요청과 취소 완료를 구분하는가?
- ☐ queued, running, UI-posted 작업을 각각 처리하는가?
- ☐ 긴 검사 단계에 취소 checkpoint가 있는가?
- ☐ blocking SDK I/O를 깨울 수 있는가?
- ☐ 새 callback 차단 → callback 종료 → worker 종료 → 자원 파괴 순서인가?
- ☐ camera handle은 SDK buffer 사용이 모두 끝난 뒤 파괴하는가?

진단

- ☐ monotonic timestamp, thread ID, camera/frame/generation을 기록하는가?
- ☐ 멈춤 시 모든 thread stack과 wait chain을 수집할 수 있는가?
- ☐ p95/p99 UI latency와 cancellation latency를 측정하는가?
- ☐ 반복·부하·종료 stress test가 있는가?

12. 핵심 결론

1. UI thread는 event loop를 돌려야 하므로 카메라 수신·검사·동기 대기를 맡기면 안 된다.
2. camera callback은 buffer 소유권을 확보해 bounded queue로 넘기고 빠르게 반환해야 한다.
3. 여러 camera는 완성된 독립 frame 값을 전달해야 하며 하나의 mutable buffer를 공유하지 않는다.
4. mutex로 data race를 없애도 여러 단계의 논리적 race, lock 역전, stale result는 별도로 해결해야 한다.
5. 가끔 멈추는 이유는 scheduler와 외부 I/O가 오류가 발생하는 interleaving을 매번 다르게 만들기 때문이다.
6. 재현이 어렵다면 log를 늘리는 것만으로 끝내지 말고 thread stack, wait chain, queue depth, tail latency를 확보한다.
7. 안전한 취소는 강제 종료가 아니라 checkpoint, buffer 수명, 결과 폐기까지 포함하는 협력적 protocol이다.
8. UI에 결과를 반영하는 마지막 순간에 lifetime과 generation을 다시 확인한다.
9. 종료할 때는 생산을 먼저 멈추고 callback 종료를 확인한 뒤 소비자를 drain/cancel하고 자원을 파괴한다.
10. camera SDK의 buffer retain·반납·callback 종료 계약은 일반 C++ 원칙만으로 추론하지 말고 제조사 문서로 확인한다.

13. 작성 후 검증 기록

13.1 Markdown과 C++ 검증 항목

- 요청한 다섯 주제가 각각 독립된 절과 통합 예제에 포함되었는지 확인한다.
- 코드 fence, 내부 링크, UTF-8 문자, 미완성 표시를 검사한다.
- C++20 예제를 warning 수준을 높여 컴파일한다.
- 여러 camera frame 제출·UI 전달·취소·종료를 반복 실행한다.

13.2 PDF 1차 대조

문서 내용	PDF에서 확인한 근거	적용 범위
실행마다 thread 순서가 달라 race가 간헐적으로 드러남	cpp.pdf 물리 744~748쪽	직접 근거
mutex 임계 영역과 unlock 누락 시 진행 정지	cpp.pdf 물리 749~753쪽	직접 근거
여러 lock은 정해진 순서로 획득	cpp.pdf 물리 754~759쪽	직접 근거
queue, mutex, CV, stop, notify, join으로 worker 관리	cpp.pdf 물리 812~817쪽	직접 근거
생산자-소비자는 shared state와 predicate로 대기	ostep.pdf 물리 348~359쪽	직접 근거
blocking call은 event loop 전체를 멈출 수 있음	ostep.pdf 물리 398~403쪽	직접 근거
thread는 주소 공간을 공유하며 동기화 문제가 발생	ocs.pdf 스레드와 병행성 장	직접 근거
mutex:semaphore와 deadlock 조건·예방	ocs.pdf 동기화와 교착 상태 장	직접 근거
camera callback 복사, generation, UI dispatcher	세 PDF에 특정 SDK/UI 구현 없음	PDF 원칙을 다중 카메라 시스템에 적용한 설계 종합

13.3 2차 검증 시 확인할 경계

- PDF가 설명하는 일반 동시성 원리와 특정 camera SDK 계약을 혼동하지 않는다.
- “분리하면 무조건 빨라진다”가 아니라 응답성·격리·소유권이 좋아진다고 설명한다.
- 취소 요청이 즉시 취소 완료를 뜻하지 않음을 유지한다.
- queue full에서 frame을 버려도 되는 업무와 버리면 안 되는 업무를 구분한다.
- camera acquisition 중지와 camera object 파괴 순서를 구분한다.

13.4 2차 PDF-Markdown-실행 검증 결과

재검증 항목	다른 근거 페이지로 다시 확인한 내용	결과
lock과 교착 상태	cpp.pdf 물리 753쪽의 반대 lock 순서 예제	문서의 ABBA 설명과 일치
worker 대기-종료	cpp.pdf 물리 814~815쪽의 queue predicate, stop, notify, join	bounded queue 종료 설명과 일치
조건 변수 predicate	ostep.pdf 물리 360쪽의 while 재검사와 spurious wake-up 주의	문서 설명과 일치
event 기반 처리의 한계	ostep.pdf 물리 403쪽의 blocking-paging-멀티코어 문제	UI event loop 주의와 일치
명령 재정렬과 동기화	ocs.pdf 물리 235쪽의 공유 flag/data 순서 문제	단순 flag 공유 경고와 일치
busy waiting과 semaphore	ocs.pdf 물리 243쪽의 spinlock-wait/signal 설명	대기 방식 구분과 일치

최종 Markdown 검사 결과:

- UTF-8 replacement character: 0개
- code fence: 48개, 짝 일치
- 내부 문서 링크: 누락 0개
- 요청한 다섯 필수 주제: 누락 0개
- 미완성 작업 표식: 0개

통합 C++ 예제는 MSVC에서 `/std:c++20 /EHsc /W4 /permissive-` 로 컴파일했다. 두 camera frame의 독립 처리, UI queue에 들어간 stale 결과의 취소, worker 종료를 포함한 실행을 300회 반복해 모두 정상 종료함을 확인했다.